
UNIT 13 TEXT PROCESSING

Structure

- 13.0 Introduction
- 13.1 Objectives
- 13.2 Abundance of Digitized Text
- 13.3 Pattern-Matching Algorithms
- 13.4 Dynamic Programming
- 13.5 Text Compression and the Greedy Method
- 13.6 Tries
- 13.7 Summary
- 13.8 Solutions/ Answer to Check your Progress
- 13.9 References / Further Reading

13.0 Introduction

Text processing has become one of the most important areas of computer science due to the exponential growth of digital text data. From search engines processing billions of web pages to DNA sequencing analyzing genetic codes, efficient text processing algorithms are fundamental to modern computing applications.

Text processing encompasses various computational problems including pattern matching, text compression, information retrieval, natural language processing, and bioinformatics. These applications require sophisticated algorithms that can handle large volumes of text data efficiently while providing accurate and fast results.

This unit explores fundamental text processing algorithms and data structures. We examine pattern-matching algorithms for finding occurrences of patterns within text, dynamic programming approaches for sequence alignment and edit distance calculations, compression techniques using greedy algorithms, and specialized data structures like tries for efficient string storage and retrieval.

The algorithms and techniques covered in this unit form the foundation for more advanced applications in search engines, data compression tools, spell checkers, DNA analysis software, and machine translation systems.

13.1 Objectives

After completing this unit, you will be able to:

- Understand the significance and challenges of processing large-scale digitized text
- Implement various pattern-matching algorithms including brute force, Boyer-Moore, and Knuth-Morris-Pratt
- Apply dynamic programming techniques to solve text processing problems like edit distance and longest common subsequence
- Design and implement text compression algorithms using greedy methods and Huffman coding
- Create and manipulate trie data structures for efficient string storage and pattern matching
- Analyze time and space complexity of different text processing algorithms
- Choose appropriate algorithms based on specific text processing requirements

- Apply text processing techniques to real-world problems in bioinformatics, information retrieval, and data compression

13.2 Abundance of Digitized Text

The amount of digitized text has grown exponentially over the past decades. Consider these statistics:

- Google processes over 8.5 billion searches per day
- Social media platforms generate petabytes of text daily
- DNA databases contain billions of nucleotide sequences
- Digital libraries store millions of books and documents

Growth of Digital Text Data:

Year Estimated Digital Text (Petabytes)

2010	~1
2015	~10
2020	~100
2025	~1000 (projected)

CHALLENGES IN TEXT PROCESSING

Scale Challenges:

- **Volume:** Processing terabytes of text data
- **Velocity:** Real-time processing requirements
- **Variety:** Multiple languages, formats, and encodings

Algorithmic Challenges:

- **Efficiency:** $O(n)$ or $O(n \log n)$ algorithms preferred over $O(n^2)$
- **Memory:** Working with datasets larger than available RAM
- **Accuracy:** Handling noise, errors, and variations in text

TEXT REPRESENTATION AND ENCODING

Understanding text encoding is crucial for efficient processing:

Algorithm Explanation:

Text encoding is like having different languages for computers to understand human text. Just as humans use different alphabets (Latin, Cyrillic, Chinese), computers use different encoding schemes to represent text.

The Fundamental Problem: Computers only understand numbers (0s and 1s), but humans work with letters, symbols, and characters. We need a systematic way to convert between human text and computer numbers.

Historical Evolution of Encoding:

ASCII (American Standard Code for Information Interchange):

Purpose: The original encoding system for English text

Range: Uses 7 bits to represent 128 characters (0-127)

Coverage: English letters, digits, punctuation, and control characters

Limitation: Only works for English and basic symbols

Unicode Revolution:

Purpose: A universal encoding system for all world languages

Goal: Assign a unique number (code point) to every character in every language

Coverage: Over 1 million possible characters including emojis, ancient scripts, mathematical symbols

Common Unicode Encodings:

UTF-8 (8-bit Unicode Transformation Format):

Smart design: Uses 1-4 bytes per character depending on what's needed

ASCII compatible: First 128 characters match ASCII exactly

Variable length: Common characters use fewer bytes, rare ones use more

UTF-16 and UTF-32:

UTF-16: Uses 2 or 4 bytes per character

UTF-32: Uses exactly 4 bytes per character

Tradeoff: More predictable but potentially wasteful of space

Text Encoding Algorithm Steps:

Step 1: Character Analysis

Identify the character: Look at each character in the text

Determine code point: Find the Unicode number assigned to this character

Check character type: Is it ASCII, Latin-1, or requires full Unicode?

Step 2: Choose Encoding Scheme

Analyze text composition: What types of characters are present?

Consider space efficiency: How much storage is available?

Think about compatibility: What systems will read this text?

Step 3: Apply Encoding Rules

For UTF-8 encoding:

ASCII characters (0-127): Use 1 byte as-is

Extended characters (128-2047): Use 2 bytes

Most other characters: Use 3 bytes

Rare/special characters: Use 4 bytes

Step 4: Generate Byte Sequence

Convert each character: Apply the encoding rules to get bytes

Concatenate results: Combine all bytes into final encoded text

Add metadata if needed: Some formats include encoding information

Visual Encoding Diagrams:

ASCII Encoding Example:

Character: 'A'

ASCII Code: 65

Binary: 01000001

Bytes needed: 1

Storage: [01000001]

UTF-8 Encoding Examples:

Character: 'A' (ASCII)

Unicode: U+0041 (65)

UTF-8: 01000001

Bytes: [01000001] (1 byte)

Character: 'é' (Latin Extended)

Unicode: U+00E9 (233)

UTF-8: 11000011 10101001

Bytes: [11000011, 10101001] (2 bytes)

Character: '世' (Chinese)

Unicode: U+4E16 (19990)

UTF-8: 11100100 10111000 10010110

Bytes: [11100100, 10111000, 10010110] (3 bytes)

Practical Encoding Algorithm:

```

def analyze_text_encoding_needs(text):
    """
    Algorithm to determine optimal encoding for given text.

    Step-by-step process:
    1. Analyze character types in the text
    2. Calculate storage requirements for different encodings
    3. Recommend best encoding choice
    4. Show detailed breakdown of encoding process
    """

    print(f"=== Text Encoding Analysis for: '{text}' ===")

    # Step 1: Character Analysis
    char_analysis = {
        'ascii_count': 0, # Characters 0-127
        'latin1_count': 0, # Characters 128-255
        'unicode_bmp_count': 0, # Characters 256-65535 (Basic Multilingual Plane)
        'unicode_extended_count': 0, # Characters > 65535
        'total_chars': len(text)
    }

    print("\nStep 1: Character Analysis")
    print("Character | Unicode | Type | UTF-8 Bytes Needed")
    print("-" * 50)

    for char in text:
        code_point = ord(char)

        if code_point <= 127:
            char_type = "ASCII"
            char_analysis['ascii_count'] += 1
            utf8_bytes = 1
        elif code_point <= 255:
            char_type = "Latin-1"
            char_analysis['latin1_count'] += 1
            utf8_bytes = 2
        elif code_point <= 65535:
            char_type = "Unicode BMP"
            char_analysis['unicode_bmp_count'] += 1
            utf8_bytes = 3
        else:
            char_type = "Unicode Extended"
            char_analysis['unicode_extended_count'] += 1
            utf8_bytes = 4

    print(f"  {char}    | U+{code_point:04X} | {char_type:12} | {utf8_bytes}")

    # Step 2: Calculate storage requirements
    print(f"\nStep 2: Storage Requirements Calculation")

```

```

# ASCII encoding (only works if all characters are ASCII)
ascii_possible = char_analysis['latin1_count'] == 0 and \
char_analysis['unicode_bmp_count'] == 0 and \
char_analysis['unicode_extended_count'] == 0

ascii_bytes = len(text) if ascii_possible else "Not possible"

# UTF-8 encoding
utf8_bytes = (char_analysis['ascii_count'] * 1 +
char_analysis['latin1_count'] * 2 +
char_analysis['unicode_bmp_count'] * 3 +
char_analysis['unicode_extended_count'] * 4)

# UTF-16 encoding
utf16_bytes = (char_analysis['ascii_count'] * 2 +
char_analysis['latin1_count'] * 2 +
char_analysis['unicode_bmp_count'] * 2 +
char_analysis['unicode_extended_count'] * 4)

# UTF-32 encoding
utf32_bytes = len(text) * 4

print(f'ASCII: {ascii_bytes} bytes')
print(f'UTF-8: {utf8_bytes} bytes')
print(f'UTF-16: {utf16_bytes} bytes')
print(f'UTF-32: {utf32_bytes} bytes')

# Step 3: Encoding recommendation
print(f"\nStep 3: Encoding Recommendation")

if ascii_possible:
    recommended = "ASCII or UTF-8"
    reason = "All characters are ASCII-compatible"
elif char_analysis['unicode_bmp_count'] == 0 and
char_analysis['unicode_extended_count'] == 0:
    recommended = "UTF-8"
    reason = "Mix of ASCII and Latin characters - UTF-8 is most efficient"
elif utf8_bytes <= utf16_bytes:
    recommended = "UTF-8"
    reason = "UTF-8 provides better compression for this text"
else:
    recommended = "UTF-16"
    reason = "Many non-ASCII characters - UTF-16 might be more
efficient"

print(f'Recommended encoding: {recommended}')
print(f'Reason: {reason}')

return {
    'analysis': char_analysis,
    'storage_requirements': {
        'ascii': ascii_bytes,
        'utf8': utf8_bytes,

```

```

        'utf16': utf16_bytes,
        'utf32': utf32_bytes
    },
    'recommendation': recommended
}

def demonstrate_utf8_encoding_process(text):
    """
    Show step-by-step UTF-8 encoding process for each character.
    """
    print(f"\n=== UTF-8 Encoding Process for: '{text}' ===")

    result_bytes = []

    for i, char in enumerate(text):
        code_point = ord(char)
        print(f"\nCharacter {i+1}: '{char}'")
        print(f"Unicode code point: U+{code_point:04X} ({code_point})")

        if code_point <= 0x7F: # 1-byte sequence
            utf8_bytes = [code_point]
            print(f"Range: 0x00-0x7F (ASCII)")
            print(f"UTF-8 pattern: 0xxxxxxx")
            print(f"Binary: {code_point:08b}")
            print(f"UTF-8 bytes: [{code_point:02X}]")

        elif code_point <= 0x7FF: # 2-byte sequence
            byte1 = 0b11000000 | (code_point >> 6)
            byte2 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2]
            print(f"Range: 0x80-0x7FF (Latin Extended)")
            print(f"UTF-8 pattern: 110xxxxx 10xxxxxx")
            print(f"Binary: {code_point:011b}")
            print(f"Split: {code_point >> 6:05b} + {code_point & 0b00111111:06b}")
            print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}]")

        elif code_point <= 0xFFFF: # 3-byte sequence
            byte1 = 0b11100000 | (code_point >> 12)
            byte2 = 0b10000000 | ((code_point >> 6) & 0b00111111)
            byte3 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2, byte3]
            print(f"Range: 0x800-0xFFFF (Basic Multilingual Plane)")
            print(f"UTF-8 pattern: 1110xxxx 10xxxxxx 10xxxxxx")
            print(f"Binary: {code_point:016b}")
            print(f"Split: {code_point >> 12:04b} + {(code_point >> 6) & 0b00111111:06b} + {code_point & 0b00111111:06b}")
            print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}, {byte3:02X}]")

        else: # 4-byte sequence
            byte1 = 0b11110000 | (code_point >> 18)
            byte2 = 0b10000000 | ((code_point >> 12) & 0b00111111)
            byte3 = 0b10000000 | ((code_point >> 6) & 0b00111111)
            byte4 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2, byte3, byte4]

```

```

print(f"Range: 0x10000+ (Extended Unicode)")
print(f"UTF-8 pattern: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx")
print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}, {byte3:02X},
{byte4:02X}]")

result_bytes.extend(utf8_bytes)

print(f"\nFinal UTF-8 encoding: {[f'{b:02X}' for b in result_bytes]}")
print(f"Total bytes needed: {len(result_bytes)}")

    return result_bytes

def create_encoding_comparison_chart():
    """
    Create a visual comparison of different encoding schemes.
    """
    print("\n=== Encoding Comparison Chart ===")

    test_cases = [
        ("Hello", "English text"),
        ("Café", "French text with accents"),
        ("Привет", "Russian Cyrillic text"),
        ("こんにちは", "Japanese Hiragana"),
        ("🌍🚀📺", "Emojis"),
        ("Hello世界🌍", "Mixed multilingual")
    ]

    print(f'{"Text":<12} {"Description":<25} {"ASCII":<8} {"UTF-8":<8} {"UTF-16":<8} {"UTF-32":<8}')
    print("-" * 75)

    for text, description in test_cases:
        try:
            ascii_size = len(text.encode('ascii'))
        except:
            ascii_size = "N/A"

        utf8_size = len(text.encode('utf-8'))
        utf16_size = len(text.encode('utf-16'))
        utf32_size = len(text.encode('utf-32'))

        print(f'{text:<12} {description:<25} {str(ascii_size):<8} {utf8_size:<8}
{utf16_size:<8} {utf32_size:<8}')

# Demonstration of text encoding concepts
def demonstrate_text_encoding():
    """Comprehensive demonstration of text encoding concepts."""
    print("=== COMPREHENSIVE TEXT ENCODING DEMONSTRATION
===")

# Test cases with different character types
test_texts = [
    "Hello", # Pure ASCII

```

```

    "Café au lait", # ASCII + Latin-1
    "Hello世界", # ASCII + Chinese
    "🌐🔗🖨️", # Emojis (4-byte UTF-8)
]

for text in test_texts:
print("\n" + "="*60)
    result = analyze_text_encoding_needs(text)
    demonstrate_utf8_encoding_process(text)

print("\n" + "="*60)
create_encoding_comparison_chart()

# Run the demonstration
demonstrate_text_encoding()
import sys
import unicodedata

class TextAnalyzer:
    """Class for analyzing text properties and encodings."""

    def __init__(self, text):
        """Initialize with text string."""
        self.text = text
        self.length = len(text)

    def analyze_encoding(self):
        """Analyze text encoding properties."""
        print(f'Text: '{self.text}''')
        print(f'Length: {self.length} characters')
        print(f'Memory size: {sys.getsizeof(self.text)} bytes')

# Character analysis
char_types = {
    'ascii': 0,
    'latin': 0,
    'unicode': 0,
    'digits': 0,
    'spaces': 0,
    'punctuation': 0
}

for char in self.text:
    if ord(char) < 128:
        char_types['ascii'] += 1
    elif ord(char) < 256:
        char_types['latin'] += 1
    else:
        char_types['unicode'] += 1

    if char.isdigit():
        char_types['digits'] += 1
    elif char.isspace():
        char_types['spaces'] += 1

```

```

elif not char.isalnum():
    char_types['punctuation'] += 1

print("Character distribution:")
for char_type, count in char_types.items():
    if count > 0:
        percentage = (count / self.length) * 100
print(f' {char_type}: {count} ({percentage:.1f}%)')

def get_encoding_sizes(self):
    """Compare different encoding sizes."""
    encodings = ['utf-8', 'utf-16', 'utf-32', 'ascii']
    sizes = {}

    for encoding in encodings:
        try:
            encoded = self.text.encode(encoding)
            sizes[encoding] = len(encoded)
        except UnicodeEncodeError:
            sizes[encoding] = "Cannot encode"

print("\nEncoding sizes (bytes):")
for encoding, size in sizes.items():
    print(f' {encoding}: {size}')

    return sizes

# Demonstrate text analysis
def demonstrate_text_analysis():
    """Demonstrate text analysis and encoding."""
    print("=== Text Analysis Demonstration ===")

# Sample texts in different languages
texts = [
    "Hello, World!",
    "Café au lait",
    "こんにちは世界",
    "DNA sequence: ATCGATCG",
    "Mixed: Hello 世界 123!"
]

for text in texts:
    print(f"\n--- Analysis for: '{text}' ---")
    analyzer = TextAnalyzer(text)
    analyzer.analyze_encoding()
    analyzer.get_encoding_sizes()

demonstrate_text_analysis()

```

APPLICATIONS OF TEXT PROCESSING

Search Engines:

- Web crawling and indexing
- Query processing and ranking

- Snippet generation

Bioinformatics:

- DNA/RNA sequence analysis
- Protein structure prediction
- Genome assembly

Natural Language Processing:

- Machine translation
- Sentiment analysis
- Named entity recognition

Data Compression:

- File compression utilities
- Database compression
- Network data transmission

13.3 Pattern-Matching Algorithms

Pattern matching is fundamental to text processing - finding all occurrences of a pattern string P within a text string T.

Pattern Matching Visualization:

Text: "ABAAABCDABABCABCABCDAB"

Pattern: "ABCD"

Matches: positions 5 and 17

```
T: A B A A A B C D A B A B C A B C A B C D A B
P:   A B C D           A B C D
    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
```

BRUTE FORCE ALGORITHM

The simplest approach checks every possible position:

Algorithm Explanation:

The brute force pattern matching algorithm works like reading a book and looking for a specific word:

1. **Start at the beginning:** Place your finger at the first character of the text
2. **Try to match:** Compare each character of the pattern with the text starting from your current position
3. **If all characters match:** You found the pattern! Record the position
4. **If any character doesn't match:** Move your finger one position forward in the text and try again
5. **Repeat:** Continue this process until you reach the end of the text

If you're looking for the word "HELLO" in a sentence. You start at the first letter, check if "H-E-L-L-O" matches the next 5 characters. If not, you move one letter forward and check again. This continues until you've checked every possible starting position.

Why is it called "brute force"?: Because it doesn't use any clever tricks - it simply tries every possibility, even when it's obvious that some positions can't work.

```
def brute_force_search(text, pattern):
    """
    Brute force pattern matching algorithm.
    Time Complexity: O(nm) where n = len(text), m = len(pattern)
    Space Complexity: O(1)
```

```

"""
n, m = len(text), len(pattern)
matches = []

# Check each possible starting position
for i in range(n - m + 1):
    # Check if pattern matches at position i
    match = True
    for j in range(m):
        if text[i + j] != pattern[j]:
            match = False
            break

    if match:
matches.append(i)

return matches

def brute_force_search_detailed(text, pattern):
    """Brute force with detailed step-by-step output."""
    n, m = len(text), len(pattern)
    matches = []
    comparisons = 0

    print(f'Searching for pattern '{pattern}' in text '{text}')
    print(f'Text length: {n}, Pattern length: {m}')
    print()

    for i in range(n - m + 1):
        print(f'Position {i}:')
        print(f'Text: {text}')
        print(f'Pattern: {' ' * i} {pattern}')

        match = True
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
        print(f'{' ' * (i + j)} ^ Mismatch at position {j}')
        break
        j += 1

        if match:
matches.append(i)
        print(f'{' ' * i} {'✓' * m} MATCH found!")

    print()

    print(f'Total comparisons: {comparisons}')
    print(f'Matches found at positions: {matches}')
    return matches

# Demonstrate brute force algorithm

```

```

def demonstrate_brute_force():
    """Demonstrate brute force pattern matching."""
    print("=== Brute Force Pattern Matching ===")

    test_cases = [
        ("ABAAABCDABABCABCABCDAB", "ABCD"),
        ("AAAAAAAAAAAA", "AAA"),
        ("HELLO WORLD", "WORLD"),
        ("ABCDEF", "XYZ")
    ]

    for text, pattern in test_cases:
        print(f"\n--- Test Case ---")
        matches = brute_force_search_detailed(text, pattern)
        print(f"Result: {len(matches)} matches found")
        print("-" * 50)

    demonstrate_brute_force()

```

KNUTH-MORRIS-PRATT (KMP) ALGORITHM

KMP algorithm avoids redundant comparisons by preprocessing the pattern to create a failure function:

Algorithm Explanation:

The KMP algorithm is like having a smart bookmark system when searching for a pattern:

Phase 1 - Preprocessing (Building the LPS Array):

1. **Analyze the pattern first:** Before searching, examine the pattern to find places where it repeats itself
2. **Create a "failure function":** For each position in the pattern, record how many characters you can skip if a mismatch occurs
3. **The key insight:** If you've already matched some characters and then find a mismatch, you don't need to start over completely - you can use previously matched characters

Phase 2 - Searching:

1. **Start comparing:** Compare text and pattern characters one by one
2. **When characters match:** Move forward in both text and pattern
3. **When mismatch occurs:** Instead of starting over, use the failure function to determine how far back to move in the pattern
4. **Never go backward in text:** This is the key efficiency - we never re-examine text characters

Real-world analogy: It's like proofreading a document. If you're checking for the word "ABABAB" and you've matched "ABAB" but the next character doesn't match, you don't start over. You realize that the "AB" you just matched could be the start of a new attempt, so you continue from there.

Why is it faster?: Because it eliminates redundant work by remembering what it has already learned about the pattern.

```

def compute_lps(pattern):
    """
    Compute Longest Prefix Suffix (LPS) array for KMP algorithm.
    LPS[i] = length of longest proper prefix which is also suffix for pattern[0:i+1]
    """
    m = len(pattern)

```

```

lps = [0] * m
length = 0 # length of previous longest prefix suffix
i = 1

print(f"Computing LPS array for pattern: '{pattern}'")
print(f"Initial LPS: {lps}")

while i < m:
    if pattern[i] == pattern[length]:
        length += 1
    lps[i] = length
    print(f"Match at {i}: pattern[{i}] = pattern[{length-1}], LPS[{i}] = {length}")
    i += 1
    else:
        if length != 0:
            length = lps[length - 1]
        print(f"Mismatch at {i}: backtrack length to {length}")
        else:
            lps[i] = 0
        print(f"No prefix match at {i}: LPS[{i}] = 0")
        i += 1

print(f"Final LPS array: {lps}")
return lps

def kmp_search(text, pattern):
    """
    Knuth-Morris-Pratt pattern matching algorithm.
    Time Complexity: O(n + m)
    Space Complexity: O(m)
    """
    n, m = len(text), len(pattern)
    if m == 0:
        return []

    # Preprocess pattern to get LPS array
    lps = compute_lps(pattern)

    matches = []
    i = 0 # index for text
    j = 0 # index for pattern
    comparisons = 0

    print(f"\nSearching using KMP algorithm:")
    print(f"Text: '{text}'")
    print(f"Pattern: '{pattern}'")
    print()

    while i < n:
        comparisons += 1
        print(f"Comparing text[{i}]='{text[i]}' with pattern[{j}]='{pattern[j]}'")

        if text[i] == pattern[j]:
            print(f" Match! Moving both pointers")

```

```

i += 1
    j += 1

    if j == m:
        # Found a match
        match_pos = i - j
        matches.append(match_pos)
        print(f" PATTERN FOUND at position {match_pos}!")
        j = lps[j - 1] # Get next position from LPS
        print(f" Using LPS: next j = {j}")
    elif i < n and text[i] != pattern[j]:
        # Mismatch after j matches
        if j != 0:
            j = lps[j - 1] # Don't match lps[0..lps[j-1]] chars
        print(f" Mismatch! Using LPS: j = lps[{j}] = {lps[j-1]}")
    else:
        print(f" Mismatch at start of pattern, advance text pointer")
    i += 1
print()

print(f"Total comparisons: {comparisons}")
print(f"Matches found at positions: {matches}")
return matches

# Demonstrate KMP algorithm
def demonstrate_kmp():
    """Demonstrate KMP pattern matching algorithm."""
    print("=== KMP Pattern Matching Algorithm ===")

    # Example showing KMP efficiency
    text = "ABABCABABA"
    pattern = "ABABA"

    print("First, let's compute the LPS array:")
    lps = compute_lps(pattern)

    print("\nNow, let's search for the pattern:")
    matches = kmp_search(text, pattern)

demonstrate_kmp()

```

13.4 Dynamic Programming

Dynamic programming is particularly useful for text processing problems involving sequence alignment, edit distance, and longest common subsequences.

EDIT DISTANCE (LEVENSHTEIN DISTANCE)

The edit distance between two strings is the minimum number of operations (insert, delete, substitute) needed to transform one string into another.

Algorithm Explanation:

The edit distance algorithm is like finding the cheapest way to transform one word into another using a limited set of operations:

The Problem: How do you change "CAT" into "DOG" with the minimum number of edits?

Available Operations:

1. **Insert:** Add a new character anywhere
2. **Delete:** Remove any character
3. **Substitute:** Replace one character with another

The Dynamic Programming Approach:

1. **Build a table:** Create a grid where rows represent characters of the first string and columns represent characters of the second string
2. **Fill base cases:** The first row and column represent the cost of transforming from/to an empty string (just insertions or deletions)
3. **Fill the table systematically:** For each cell, consider three possibilities:
 - **Characters match:** No operation needed, copy the diagonal value
 - **Characters don't match:** Take the minimum of:
 - Left cell + 1 (insert operation)
 - Top cell + 1 (delete operation)
 - Diagonal cell + 1 (substitute operation)

Real-world analogy: It's like finding the shortest path on a grid map where you can move right (insert), down (delete), or diagonally (substitute/match). Each move has a cost, and you want the cheapest total journey.

Why dynamic programming?: We break the problem into smaller subproblems. To transform "CAT" to "DOG", we need to know the cost of transforming "CA" to "DO", "CA" to "DOG", and "CAT" to "DO". We build up the solution from these smaller pieces.

Applications: Spell checkers, DNA sequence analysis, version control systems, plagiarism detection.

```
def edit_distance(str1, str2):
    """
    Compute edit distance using dynamic programming.
    Time Complexity: O(mn)
    Space Complexity: O(mn)
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Initialize base cases
    for i in range(m + 1):
        dp[i][0] = i # Delete all characters from str1
    for j in range(n + 1):
        dp[0][j] = j # Insert all characters from str2

    print(f"Computing edit distance between '{str1}' and '{str2}'")
    print("\nDP Table construction:")

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
                # Characters match, no operation needed
                dp[i][j] = dp[i-1][j-1]
            else:
                # Take minimum of three operations
```

```

dp[i][j] = 1 + min(
    dp[i-1][j], # Delete from str1
    dp[i][j-1], # Insert into str1
    dp[i-1][j-1] # Substitute
)

# Print DP table
print_dp_table(dp, str1, str2)

return dp[m][n]

def print_dp_table(dp, str1, str2):
    """Print DP table in formatted way."""
    m, n = len(str1), len(str2)

    # Header
    print(" ", end="")
    print("\\".rjust(3), end="")
    for char in str2:
        print(f"{char}" .rjust(3), end="")
    print()

    # Rows
    for i in range(m + 1):
        if i == 0:
            row_label = "\\"
        else:
            row_label = str1[i-1]
        print(f"{row_label}" .rjust(3), end=" ")

        for j in range(n + 1):
            print(f"{dp[i][j]}" .rjust(3), end="")
        print()

def edit_distance_with_operations(str1, str2):
    """
    Compute edit distance and track operations performed.
    Returns (distance, list of operations)
    """
    m, n = len(str1), len(str2)

    # DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Operation tracking table
    ops = ["" * (n + 1) for _ in range(m + 1)]

    # Initialize base cases
    for i in range(m + 1):
        dp[i][0] = i
        ops[i][0] = "D" * i # Delete operations
    for j in range(n + 1):
        dp[0][j] = j
        ops[0][j] = "I" * j # Insert operations

```

```

# Fill tables
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if str1[i-1] == str2[j-1]:
            dp[i][j] = dp[i-1][j-1]
            ops[i][j] = ops[i-1][j-1] + "M" # Match
        else:
            # Find minimum operation
            delete_cost = dp[i-1][j] + 1
            insert_cost = dp[i][j-1] + 1
            substitute_cost = dp[i-1][j-1] + 1

            min_cost = min(delete_cost, insert_cost, substitute_cost)
            dp[i][j] = min_cost

            if min_cost == delete_cost:
                ops[i][j] = ops[i-1][j] + "D"
            elif min_cost == insert_cost:
                ops[i][j] = ops[i][j-1] + "I"
            else:
                ops[i][j] = ops[i-1][j-1] + "S"

    return dp[m][n], ops[m][n]

def demonstrate_edit_distance():
    """Demonstrate edit distance calculation."""
    print("==== Edit Distance Demonstration ====")

    test_pairs = [
        ("CAT", "DOG"),
        ("KITTEN", "SITTING"),
        ("SUNDAY", "SATURDAY"),
        ("INTENTION", "EXECUTION")
    ]

    for str1, str2 in test_pairs:
        print(f"\n--- Edit Distance: '{str1}' → '{str2}' ---")
        distance = edit_distance(str1, str2)
        print(f"Edit distance: {distance}")

        distance_ops, operations = edit_distance_with_operations(str1, str2)
        print(f"Operations sequence: {operations}")
        print(f" M=Match, S=Substitute, I=Insert, D=Delete")

    demonstrate_edit_distance()

```

LONGEST COMMON SUBSEQUENCE (LCS)

Find the longest subsequence common to two sequences.

LCS Algorithm - High Level Steps

- **Step 1: Create DP Table**
- Build a 2D table of size $(m+1) \times (n+1)$ where m and n are the lengths of the two strings
- Initialize all cells to 0
- **Step 2: Fill the Table**

- For each cell (i,j), compare characters from both strings:
 - If characters match: take diagonal value + 1
 - If characters don't match: take maximum of left or top cell
- **Step 3: Extract Result**
- The bottom-right cell contains the LCS length
- To get the actual LCS string, backtrack from bottom-right to top-left, following the path that created the optimal solution
- **Step 4: Backtrack (for actual LCS string)**
- Start from (m,n) and work backwards
- When characters match: include character and move diagonally
- When they don't match: move toward the cell that provided the maximum value
- The algorithm builds up the solution incrementally by solving smaller subproblems and uses the results to construct the final answer.

Given below is the sample code implementation -

```
def lcs_length(str1, str2):
    """
    Compute length of Longest Common Subsequence using DP.
    Time Complexity: O(mn)
    Space Complexity: O(mn)
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    print(f"Computing LCS length between '{str1}' and '{str2}'")

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Print DP table
    print("\nLCS DP Table:")
    print_dp_table(dp, str1, str2)

    return dp[m][n]

def lcs_string(str1, str2):
    """
    Compute actual LCS string using DP with backtracking.
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
```

```

        if str1[i-1] == str2[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
        else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Backtrack to find LCS string
    lcs = []
i, j = m, n

print(f"\nBacktracking to find LCS:")
    while i > 0 and j > 0:
        if str1[i-1] == str2[j-1]:
lcs.append(str1[i-1])
print(f" Match '{str1[i-1]}' at ({i},{j}) - added to LCS")
i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
print(f" Move up from ({i},{j}) to ({i-1},{j})")
i -= 1
            else:
print(f" Move left from ({i},{j}) to ({i},{j-1})")
j -= 1

lcs.reverse()
return ".join(lcs)

def lcs_all_solutions(str1, str2):
    """Find all possible LCS solutions."""
    m, n = len(str1), len(str2)

    # Create DP table
dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
            else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Find all LCS strings using backtracking
all_lcs = set()

    def backtrack(i, j, current_lcs):
        if i == 0 or j == 0:
all_lcs.add(current_lcs[::-1]) # Reverse to get correct order
        return

        if str1[i-1] == str2[j-1]:
backtrack(i-1, j-1, current_lcs + str1[i-1])
        else:
            if dp[i-1][j] == dp[i][j-1]:
backtrack(i-1, j, current_lcs)

```

```

        if dp[i][j-1] == dp[i][j]:
            backtrack(i, j-1, current_lcs)

    backtrack(m, n, "")
    return list(all_lcs)

def demonstrate_lcs():
    """Demonstrate LCS computation."""
    print("=== Longest Common Subsequence Demonstration ===")

    test_pairs = [
        ("ABCDGH", "AEDFHR"),
        ("AGGTAB", "GXTXAYB"),
        ("PROGRAMMING", "CRAMMING"),
        ("ABCDEFGH", "ACEG")
    ]

    for str1, str2 in test_pairs:
        print(f"\n--- LCS: '{str1}' and '{str2}' ---")
        length = lcs_length(str1, str2)
        print(f"LCS length: {length}")

        lcs_str = lcs_string(str1, str2)
        print(f"LCS string: '{lcs_str}'")

        all_solutions = lcs_all_solutions(str1, str2)
        print(f"All LCS solutions: {all_solutions}")

    demonstrate_lcs()

```

13.5 Text Compression and the Greedy Method

Text compression reduces storage space and transmission time. Greedy algorithms like Huffman coding provide optimal prefix-free codes.

HUFFMAN CODING

Huffman coding assigns shorter codes to more frequent characters.

Huffman Coding Algorithm - High Level Steps

- **Step 1: Build Frequency Table**
 - Count the frequency of each character in the input text
 - Create a frequency table mapping each character to its occurrence count
- **Step 2: Create Initial Priority Queue**
 - Create a leaf node for each character with its frequency
 - Add all leaf nodes to a min-heap (priority queue) ordered by frequency
- **Step 3: Build Huffman Tree**
 - While there's more than one node in the heap:
 - Extract the two nodes with lowest frequencies
 - Create a new internal node with these as children
 - Set the internal node's frequency as the sum of its children's frequencies
 - Add the internal node back to the heap
 - The remaining node becomes the root of the Huffman tree
- **Step 4: Generate Codes**

- Traverse the tree from root to each leaf
- Assign '0' for left edges and '1' for right edges
- The path from root to leaf becomes that character's binary code
- More frequent characters end up with shorter codes
- **Step 5: Encode Text**
- Replace each character in the original text with its corresponding Huffman code
- Concatenate all codes to create the compressed binary string
- **Step 6: Decode Text (when needed)**
- Traverse the Huffman tree bit by bit using the encoded string
- When reaching a leaf node, output the character and return to root
- Continue until all bits are processed
- The algorithm uses a greedy approach to build an optimal prefix-free code where more frequent characters get shorter codes, resulting in overall compression.

Given below is the code implementation -

```
import heapq
from collections import defaultdict, Counter

class HuffmanNode:
    """Node class for Huffman tree."""

    def __init__(self, char=None, freq=0, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):
        """For heap comparison."""
        return self.freq < other.freq

    def __repr__(self):
        if self.char:
            return f'HuffmanNode('{self.char}': {self.freq})'
        else:
            return f'HuffmanNode(internal: {self.freq})'

class HuffmanCoding:
    """Huffman coding implementation for text compression."""

    def __init__(self):
        self.codes = {}
        self.reverse_codes = {}
        self.root = None

    def build_frequency_table(self, text):
        """Build frequency table for characters in text."""
        frequency = Counter(text)
        print(f'Character frequencies:')
        for char, freq in sorted(frequency.items()):
            print(f' '{char}': {freq}')
```

```

        return frequency

    def build_huffman_tree(self, frequency):
        """Build Huffman tree using greedy algorithm."""
        if len(frequency) == 1:
            # Special case: only one unique character
            char = list(frequency.keys())[0]
            self.root = HuffmanNode(char, frequency[char])
            return self.root

        # Create priority queue with leaf nodes
        heap = []
        for char, freq in frequency.items():
            node = HuffmanNode(char, freq)
            heapq.heappush(heap, node)

        print(f"\nBuilding Huffman tree:")
        step = 1

        # Build tree bottom-up
        while len(heap) > 1:
            # Extract two nodes with minimum frequency
            left = heapq.heappop(heap)
            right = heapq.heappop(heap)

            # Create internal node
            internal_freq = left.freq + right.freq
            internal_node = HuffmanNode(None, internal_freq, left, right)

            print(f"Step {step}: Merge {left} + {right} = {internal_node}")
            step += 1

            # Add internal node back to heap
            heapq.heappush(heap, internal_node)

        self.root = heap[0]
        return self.root

    def generate_codes(self, node=None, code=""):
        """Generate Huffman codes for each character."""
        if node is None:
            node = self.root

        if node.char: # Leaf node
            if code == "": # Single character case
                code = "0"
            self.codes[node.char] = code
            self.reverse_codes[code] = node.char
            print(f" '{node.char}': {code}")
        else: # Internal node
            if node.left:
                self.generate_codes(node.left, code + "0")
            if node.right:
                self.generate_codes(node.right, code + "1")

```

```

def encode(self, text):
    """Encode text using Huffman codes."""
    if not self.codes:
        frequency = self.build_frequency_table(text)
    self.build_huffman_tree(frequency)
    print(f"\nGenerated Huffman codes:")
    self.generate_codes()

    encoded = ""
    for char in text:
        encoded += self.codes[char]

    return encoded

def decode(self, encoded_text):
    """Decode Huffman encoded text."""
    if not self.reverse_codes:
        raise ValueError("No codes available for decoding")

    decoded = ""
    current_code = ""

    for bit in encoded_text:
        current_code += bit
        if current_code in self.reverse_codes:
            decoded += self.reverse_codes[current_code]
        current_code = ""

    if current_code: # Incomplete code
        raise ValueError(f"Incomplete code: {current_code}")

    return decoded

def get_compression_stats(self, original_text, encoded_text):
    """Calculate compression statistics."""
    original_bits = len(original_text) * 8 # Assuming 8 bits per character
    compressed_bits = len(encoded_text)
    compression_ratio = compressed_bits / original_bits
    space_saved = 1 - compression_ratio

    return {
        'original_bits': original_bits,
        'compressed_bits': compressed_bits,
        'compression_ratio': compression_ratio,
        'space_saved': space_saved,
        'avg_bits_per_char': compressed_bits / len(original_text)
    }

def display_tree(self, node=None, level=0, prefix="Root: "):
    """Display Huffman tree structure."""
    if node is None:
        node = self.root

```

```

        if node:
            if node.char:
                print(" " * (level * 4) + prefix + f"{node.char}' (freq: {node.freq})")
            else:
                print(" " * (level * 4) + prefix + f"Internal (freq: {node.freq})")
                if node.left or node.right:
                    self.display_tree(node.left, level + 1, "L--- ")
                    self.display_tree(node.right, level + 1, "R--- ")

def demonstrate_huffman_coding():
    """Demonstrate Huffman coding algorithm."""
    print("=== Huffman Coding Demonstration ===")

    # Test text with varying character frequencies
    text = "ABRACADABRA"

    print(f"Original text: '{text}'")
    print(f"Length: {len(text)} characters")

    # Create Huffman coder and encode
    huffman = HuffmanCoding()
    encoded = huffman.encode(text)

    print(f"\nHuffman tree structure:")
    huffman.display_tree()

    print(f"\nEncoded text: {encoded}")
    print(f"Encoded length: {len(encoded)} bits")

    # Decode and verify
    decoded = huffman.decode(encoded)
    print(f"Decoded text: '{decoded}'")
    print(f"Decoding successful: {decoded == text}")

    # Compression statistics
    stats = huffman.get_compression_stats(text, encoded)
    print(f"\nCompression Statistics:")
    print(f" Original size: {stats['original_bits']} bits")
    print(f" Compressed size: {stats['compressed_bits']} bits")
    print(f" Compression ratio: {stats['compression_ratio']:.3f}")
    print(f" Space saved: {stats['space_saved']:.1%}")
    print(f" Average bits per character: {stats['avg_bits_per_char']:.2f}")

demonstrate_huffman_coding()

```

RUN-LENGTH ENCODING

Simple compression for data with many consecutive repeated values

Run-Length Encoding Algorithm - High Level Steps

Encoding Process

Step 1: Initialize Variables

- Start with the first character as the current character
- Set count to 1
- Create an empty result list

Step 2: Scan Through Text

- For each subsequent character in the input:
 - If it matches the current character: increment the count
 - If it's different: record the current character and its count, then reset for the new character

Step 3: Record Final Run

- After scanning all characters, don't forget to record the last character and its count

Step 4: Format Output

- Combine each character with its count (e.g., "A3B2C1")
- Join all runs into the final encoded string

Decoding Process

Step 1: Initialize Parsing

- Start at the beginning of the encoded string
- Create an empty result list

Step 2: Parse Character-Count Pairs

- Read each character
- Read the following digits to get the count (handle multi-digit numbers)
- Repeat the character that many times

Step 3: Reconstruct Text

- Concatenate all repeated character sequences
- Return the original text

Key Characteristics

- **Best for:** Data with many consecutive repeated characters
- **Compression efficiency:** Depends on repetition patterns - can actually increase size if there are few repeats
- **Simple and fast:** Linear time complexity $O(n)$
- **Lossless:** Perfect reconstruction of original data

The algorithm works by replacing runs of identical consecutive characters with a single character followed by the count of repetitions.

Sample implementation is given below -

```
class RunLengthEncoding:
    """Run-Length Encoding implementation."""

    @staticmethod
    def encode(text):
        """
        Encode text using run-length encoding.
        Format: character followed by count
        """
        if not text:
            return ""

        encoded = []
        current_char = text[0]
        count = 1

        print(f'Encoding text: '{text}')
        print("Step-by-step encoding:")

        for i in range(1, len(text)):
            if text[i] == current_char:
                count += 1
```

```

print(f' Position {i}: '{text[i]}' matches, count = {count}')
    else:
        # Different character found, record current run
        encoded.append(f'{current_char} {count}')
        print(f' Position {i}: '{text[i]}' differs, record '{current_char} {count}''')
        current_char = text[i]
        count = 1

    # Don't forget the last run
    encoded.append(f'{current_char} {count}')
    print(f' End: record final run '{current_char} {count}''')

    result = ".join(encoded)
    print(f'Encoded result: '{result}''')
    return result

    @staticmethod
    def decode(encoded_text):
        """
        Decode run-length encoded text.
        """
        if not encoded_text:
            return ""

        decoded = []
        i = 0

        print(f'Decoding text: '{encoded_text}''')
        print("Step-by-step decoding:")

        while i < len(encoded_text):
            char = encoded_text[i]
            i += 1

            # Read count (may be multiple digits)
            count_str = ""
            while i < len(encoded_text) and encoded_text[i].isdigit():
                count_str += encoded_text[i]
                i += 1

            count = int(count_str)
            decoded_part = char * count
            decoded.append(decoded_part)

        print(f' '{char}' × {count} = '{decoded_part}''')

        result = ".join(decoded)
        print(f'Decoded result: '{result}''')
        return result

    def demonstrate_rle():
        """Demonstrate Run-Length Encoding."""
        print("=== Run-Length Encoding Demonstration ===")

```

```

test_strings = [
    "AAABBBCCDAAA",

    "WWWWWWWWWWWWBWWWWWWWWWWWWBBBWWWWWWWW
    WWWWWWWWWWWWWWWWWBWWWWWWWWWWWWWW",
    "ABCDEF", # No repetition
    "AAAAAAAAAA" # All same character
]

rle = RunLengthEncoding()

for text in test_strings:
    print(f'\n--- Test String: '{text}' ---')
    print(f'Original length: {len(text)}')

    # Encode
    encoded = rle.encode(text)
    print(f'Encoded length: {len(encoded)}')

    # Calculate compression ratio
    if len(text) > 0:
        ratio = len(encoded) / len(text)
    print(f'Compression ratio: {ratio:.2f}')

    # Decode and verify
    decoded = rle.decode(encoded)
    print(f'Decoding successful: {decoded == text}')

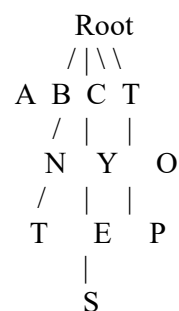
demonstrate_rle()

```

13.6 Tries

A trie (prefix tree) is a specialized tree data structure for storing strings, enabling efficient prefix-based operations.

Trie Structure Visualization:



Words: "ANT", "BYE", "BYES", "TOP"

Trie Data Structure - High Level Steps

Insert Operation

Step 1: Start at Root

- Begin traversal from the root node of the trie

Step 2: Traverse Character by Character

- For each character in the word:
 - If the character exists as a child: move to that child node
 - If the character doesn't exist: create a new child node and move to it

Step 3: Mark End of Word

- At the final character, mark the node as "end of word"
- Increment word count and trie size

Search Operation**Step 1: Start at Root**

- Begin traversal from the root node

Step 2: Follow Character Path

- For each character in the target word:
 - If character exists as a child: move to that child node
 - If character doesn't exist: return false (word not found)

Step 3: Verify Complete Word

- After traversing all characters, check if current node is marked as "end of word"
- Return true only if it's a complete word, not just a prefix

Prefix Search Operation**Step 1: Navigate to Prefix End**

- Follow the same path as search, but stop after the prefix characters

Step 2: Check Existence

- If all prefix characters found: return true
- If any character missing: return false

Delete Operation**Step 1: Verify Word Exists**

- First search to confirm the word exists in the trie

Step 2: Recursive Deletion

- Traverse to the end of the word
- Unmark "end of word" flag
- Work backwards, deleting nodes that have no other children and aren't end of other words

Step 3: Clean Up

- Remove unnecessary nodes to keep trie structure minimal

Get Words with Prefix**Step 1: Navigate to Prefix Node**

- Traverse to the node representing the end of the prefix

Step 2: Collect All Descendants

- Recursively traverse all paths from that node
- Build complete words by concatenating characters along each path
- Add words to result list when reaching "end of word" nodes

Key Characteristics

- **Time Complexity:** $O(m)$ for most operations, where m is word length
- **Space Efficient:** Shared prefixes use same nodes
- **Fast Prefix Operations:** Excellent for autocomplete and prefix matching
- **Ordered Storage:** Can retrieve words in lexicographical order

The trie efficiently stores strings by sharing common prefixes, making it ideal for applications like spell checkers, autocomplete systems, and IP routing tables.

The sample code is given below –

```
class TrieNode:
```

```

"""Node class for Trie data structure."""

def __init__(self):
    """Initialize trie node."""
self.children = {} # Dictionary mapping characters to child nodes
self.is_end_of_word = False # Marks end of a valid word
self.word_count = 0 # Number of words ending at this node

def __repr__(self):
    return f'TrieNode(children={list(self.children.keys())},
is_end={self.is_end_of_word})'

class Trie:
    """Trie (Prefix Tree) implementation for string storage and retrieval."""

    def __init__(self):
        """Initialize empty trie."""
self.root = TrieNode()
self.size = 0 # Total number of words

    def insert(self, word):
        """
        Insert a word into the trie.
        Time Complexity: O(m) where m is length of word
        """
        if not word:
            return

        current = self.root
        print(f'Inserting word: '{word}''')

        for i, char in enumerate(word):
            if char not in current.children:
                current.children[char] = TrieNode()
                print(f' Created new node for '{char}' at level {i+1}''')
            else:
                print(f' Found existing node for '{char}' at level {i+1}''')

            current = current.children[char]

        if not current.is_end_of_word:
            current.is_end_of_word = True
            self.size += 1
            print(f' Marked end of word at '{word}''')
        else:
            print(f' Word '{word}' already exists''')

        current.word_count += 1

    def search(self, word):
        """
        Search for a word in the trie.
        Time Complexity: O(m) where m is length of word
        """

```

```

        current = self.root
    print(f'Searching for word: '{word}''')

    for i, char in enumerate(word):
        if char not in current.children:
            print(f' Character '{char}' not found at level {i+1}''')
            return False
        print(f' Found '{char}' at level {i+1}''')
        current = current.children[char]

    found = current.is_end_of_word
    print(f' Word found: {found}''')
    return found

def starts_with(self, prefix):
    """
    Check if any word in trie starts with given prefix.
    Time Complexity: O(m) where m is length of prefix
    """
    current = self.root
    print(f'Checking prefix: '{prefix}''')

    for i, char in enumerate(prefix):
        if char not in current.children:
            print(f' Prefix not found: '{char}' missing at level {i+1}''')
            return False
        print(f' Found '{char}' at level {i+1}''')
        current = current.children[char]

    print(f' Prefix '{prefix}' exists''')
    return True

def delete(self, word):
    """
    Delete a word from the trie.
    Time Complexity: O(m) where m is length of word
    """
    if not self.search(word):
        print(f'Word '{word}' not found for deletion''')
        return False

    def _delete_helper(node, word, index):
        if index == len(word):
            # Reached end of word
            if not node.is_end_of_word:
                return False # Word doesn't exist

    node.is_end_of_word = False
    node.word_count -= 1
    # Return True if node has no children (can be deleted)
    return len(node.children) == 0

    char = word[index]
    child_node = node.children.get(char)

```

```

        if not child_node:
            return False # Word doesn't exist

should_delete_child = _delete_helper(child_node, word, index + 1)

        if should_delete_child:
            del node.children[char]
            # Return True if current node can also be deleted
            return (not node.is_end_of_word and
len(node.children) == 0)

        return False

print(f'Deleting word: '{word}''')
_delete_helper(self.root, word, 0)
self.size -= 1
print(f'Word '{word}' deleted successfully")
return True

def get_all_words_with_prefix(self, prefix):
    """Get all words that start with given prefix."""
    current = self.root

    # Navigate to prefix node
    for char in prefix:
        if char not in current.children:
            return [] # Prefix doesn't exist
        current = current.children[char]

    # Collect all words from this node
    words = []
self._collect_words(current, prefix, words)
    return words

def _collect_words(self, node, current_word, words):
    """Recursively collect all words from a node."""
    if node.is_end_of_word:
words.append(current_word)

        for char, child_node in node.children.items():
self._collect_words(child_node, current_word + char, words)

def get_all_words(self):
    """Get all words stored in the trie."""
    words = []
self._collect_words(self.root, "", words)
    return sorted(words)

def count_words_with_prefix(self, prefix):
    """Count words that start with given prefix."""
    current = self.root

    # Navigate to prefix node

```

```

    for char in prefix:
        if char not in current.children:
            return 0
        current = current.children[char]

    return self._count_words(current)

def _count_words(self, node):
    """Count all words from a given node."""
    count = 1 if node.is_end_of_word else 0

    for child_node in node.children.values():
        count += self._count_words(child_node)

    return count

def longest_common_prefix(self):
    """Find the longest common prefix of all words in trie."""
    if self.size == 0:
        return ""

    current = self.root
    prefix = ""

    while (len(current.children) == 1 and
           not current.is_end_of_word):
        char = list(current.children.keys())[0]
        prefix += char
        current = current.children[char]

    return prefix

def display_structure(self, node=None, level=0, char="", prefix=""):
    """Display trie structure for visualization."""
    if node is None:
        node = self.root
    print("Trie Structure:")

    # Show current node
    if char:
        indicator = " (END)" if node.is_end_of_word else ""
        print(" " * level + f"|—— '{char}'{indicator}")
    else:
        print(" " * level + "Root")

    # Show children
    children = list(node.children.items())
    for i, (child_char, child_node) in enumerate(children):
        is_last = (i == len(children) - 1)
        child_prefix = prefix + char
        self.display_structure(child_node, level + 1, child_char, child_prefix)

def demonstrate_trie():
    """Demonstrate Trie data structure operations."""

```

```

print("=== Trie Data Structure Demonstration ===")

# Create trie and insert words
trie = Trie()
words = ["CAT", "CATS", "DOG", "DOGS", "DOGGY", "CAR", "CARD",
"CARE", "CAREFUL"]

print("--- Inserting Words ---")
for word in words:
    trie.insert(word)
print()

print(f'Total words in trie: {trie.size}')

# Display trie structure
print("\n--- Trie Structure ---")
trie.display_structure()

# Search operations
print("\n--- Search Operations ---")
search_words = ["CAT", "CATCH", "DOG", "CARE"]
for word in search_words:
    found = trie.search(word)
    print(f'Search '{word}': {found}')
print()

# Prefix operations
print("--- Prefix Operations ---")
prefixes = ["CA", "DO", "XY"]
for prefix in prefixes:
    has_prefix = trie.starts_with(prefix)
    print(f'Has prefix '{prefix}': {has_prefix}')

    if has_prefix:
        words_with_prefix = trie.get_all_words_with_prefix(prefix)
        print(f'Words with prefix '{prefix}': {words_with_prefix}')
        print()

# Get all words
print("--- All Words in Trie ---")
all_words = trie.get_all_words()
print(f'All words: {all_words}')

# Longest common prefix
print(f'\nLongest common prefix: '{trie.longest_common_prefix()}'')

# Delete operations
print("\n--- Delete Operations ---")
delete_words = ["CATS", "DOGGY"]
for word in delete_words:
    trie.delete(word)
print(f'Remaining words: {trie.get_all_words()}')
print()

```

COMPRESSED TRIE (PATRICIA TREE)

A compressed trie reduces space by merging nodes with single children:

```
class CompressedTrieNode:
    """Node for compressed trie (PATRICIA tree)."""

    def __init__(self, label=""):
        """Initialize with edge label."""
    self.label = label # String label on edge to this node
    self.children = {}
    self.is_end_of_word = False
    self.words = [] # Store actual words ending at this node

    def __repr__(self):
        return f'CompressedTrieNode(label={self.label},
end={self.is_end_of_word})'

class CompressedTrie:
    """Compressed Trie (PATRICIA tree) implementation."""

    def __init__(self):
        """Initialize empty compressed trie."""
    self.root = CompressedTrieNode()
    self.size = 0

    def insert(self, word):
        """Insert word into compressed trie."""
        if not word:
            return

    print(f'Inserting word: '{word}')
        current = self.root
        remaining = word

        while remaining:
            # Find matching child
            matching_child = None
            for child in current.children.values():
                if remaining.startswith(child.label[0]):
                    matching_child = child
                    break

            if matching_child is None:
                # No matching child, create new node
                new_node = CompressedTrieNode(remaining)
                new_node.is_end_of_word = True
                new_node.words.append(word)
                current.children[remaining[0]] = new_node
                print(f' Created new node with label '{remaining}')
                break
            else:
                # Found matching child
```

```

child_label = matching_child.label
common_prefix = self._find_common_prefix(remaining, child_label)

    if common_prefix == child_label:
        # Child label is prefix of remaining string
    print(f" Traversing node with label '{child_label}'")
        current = matching_child
        remaining = remaining[len(common_prefix):]

    if not remaining:
        # Word ends at this node
        if not matching_child.is_end_of_word:
            matching_child.is_end_of_word = True
            self.size += 1
            matching_child.words.append(word)
            print(f" Word ends at existing node")

    elif common_prefix == remaining:
        # Remaining string is prefix of child label
        # Need to split the child node
    print(f" Splitting node: common='{common_prefix}', child='{child_label}'")

        # Create new intermediate node
        intermediate = CompressedTrieNode(common_prefix)
        intermediate.is_end_of_word = True
        intermediate.words.append(word)

        # Update existing child
        matching_child.label = child_label[len(common_prefix):]

        # Reconnect
        del current.children[child_label[0]]
        current.children[common_prefix[0]] = intermediate
        intermediate.children[matching_child.label[0]] = matching_child

    self.size += 1
    break

    else:
        # Partial match - need to split
    print(f" Partial match: common='{common_prefix}'")

        # Create intermediate node
        intermediate = CompressedTrieNode(common_prefix)

        # Create new leaf for remaining word
        remaining_suffix = remaining[len(common_prefix):]
        new_leaf = CompressedTrieNode(remaining_suffix)
        new_leaf.is_end_of_word = True
        new_leaf.words.append(word)

        # Update existing child
        matching_child.label = child_label[len(common_prefix):]

```

```

        # Reconnect all nodes
        del current.children[child_label[0]]
current.children[common_prefix[0]] = intermediate
intermediate.children[remaining_suffix[0]] = new_leaf
intermediate.children[matching_child.label[0]] = matching_child

self.size += 1
        break

    def _find_common_prefix(self, str1, str2):
        """Find common prefix of two strings."""
i = 0
        while (i < len(str1) and i < len(str2) and str1[i] == str2[i]):
i += 1
        return str1[:i]

    def search(self, word):
        """Search for word in compressed trie."""
        current = self.root
        remaining = word

print(f"Searching for: '{word}'")

        while remaining and current:
found_child = False

            for child in current.children.values():
                if remaining.startswith(child.label):
print(f" Matched label '{child.label}'")
                    current = child
                    remaining = remaining[len(child.label):]
found_child = True
                    break

            if not found_child:
print(f" No matching child for '{remaining}'")
                return False

        result = not remaining and current and current.is_end_of_word
print(f" Search result: {result}")
        return result

    def display_structure(self, node=None, level=0, edge_label=""):
        """Display compressed trie structure."""
        if node is None:
            node = self.root
print("Compressed Trie Structure:")
            print("Root")
        else:
end_marker = " (END)" if node.is_end_of_word else ""
words_info = f" {node.words}" if node.words else ""
print(" " * level + f" |—— '{edge_label}'{end_marker} {words_info}")

        for child in node.children.values():

```

```

self.display_structure(child, level + 1, child.label)

def demonstrate_compressed_trie():
    """Demonstrate Compressed Trie operations."""
    print("=== Compressed Trie (PATRICIA) Demonstration ===")

    # Create compressed trie
    ctrie = CompressedTrie()

    # Insert words that will demonstrate compression
    words = ["CAR", "CARD", "CARE", "CAREFUL", "CAT", "CATCH"]

    print("--- Inserting Words ---")
    for word in words:
        ctrie.insert(word)
    print()

    print(f"Total words: {ctrie.size}")

    # Display structure
    print("\n--- Compressed Trie Structure ---")
    ctrie.display_structure()

    # Search operations
    print("\n--- Search Operations ---")
    search_words = ["CAR", "CARD", "CAREER", "CAT"]
    for word in search_words:
        ctrie.search(word)
    print()

    demonstrate_compressed_trie()

```

Check Your Progress 1

- 1. Question:** What is the time complexity of the brute force pattern matching algorithm? a) $O(n + m)$ b) $O(n \times m)$ c) $O(n \log m)$ d) $O(m \log n)$
- 2. Question:** What advantage does the KMP algorithm provide over brute force pattern matching? a) Uses less memory b) Avoids redundant character comparisons c) Works only with binary strings d) Requires preprocessing of the text
- 3. Question:** In dynamic programming for edit distance, what do the three operations represent? a) Insert, delete, copy b) Insert, delete, substitute c) Match, substitute, transpose d) Copy, move, replace
- 4. Question:** What is the main principle behind Huffman coding? a) Assign equal-length codes to all characters b) Assign shorter codes to more frequent characters c) Use alphabetical ordering for code assignment d) Minimize the number of unique characters
- 5. Question:** What is the time complexity of searching for a word of length m in a trie? a) $O(\log m)$ b) $O(m \log n)$ c) $O(m)$ d) $O(n)$

- 6. Question:** Which compression technique is most suitable for data with many consecutive repeated characters? a) Huffman coding b) Run-length encoding c) LZ77 d) Arithmetic coding
- 7. Question:** What is the space complexity of computing edit distance using dynamic programming? a) $O(1)$ b) $O(m)$ c) $O(n)$ d) $O(m \times n)$
- 8. Question:** In a compressed trie (PATRICIA tree), what happens to nodes with only one child? a) They are deleted b) They are merged with their parent c) They are merged with their child d) They remain unchanged
- 9. Question:** What is the main advantage of the Boyer-Moore algorithm? a) It works from left to right b) It can skip multiple characters when a mismatch occurs c) It uses constant extra space d) It works only with ASCII characters
- 10. Question:** Which property makes Huffman codes uniquely decodable? a) All codes have the same length b) Codes are assigned alphabetically c) No code is a prefix of another code d) Codes use only 0s and 1s

13.7 Summary

Text processing algorithms and data structures are fundamental to modern computing, enabling efficient handling of the massive amounts of digital text data generated daily. This unit covered essential concepts and techniques that form the backbone of search engines, compression tools, bioinformatics applications, and natural language processing systems.

Key Concepts Covered:

- **Digitized Text Challenges:** Understanding the scale and complexity of modern text processing, including encoding issues, memory constraints, and the need for efficient algorithms that can handle terabytes of data.
- **Pattern-Matching Algorithms:** We explored multiple approaches from the simple $O(nm)$ brute force method to sophisticated algorithms like KMP $O(n+m)$, Boyer-Moore, and Rabin-Karp that use preprocessing and heuristics to achieve better average-case performance.
- **Dynamic Programming in Text Processing:** Applied DP techniques to solve fundamental problems like edit distance calculation, longest common subsequence finding, and sequence alignment - crucial for bioinformatics and spell checking applications.
- **Text Compression:** Implemented greedy algorithms including Huffman coding for optimal prefix-free encoding, run-length encoding for repetitive data, and dictionary-based methods like LZ77 that achieve significant space savings.
- **Trie Data Structures:** Explored prefix trees and compressed tries (PATRICIA trees) that enable efficient string storage, fast prefix matching, and applications like autocomplete systems and spell checkers.

Performance Summary:

- Pattern matching: $O(n+m)$ with KMP, $O(n/m)$ average with Boyer-Moore
- Edit distance: $O(mn)$ time and space complexity
- Huffman coding: $O(n \log n)$ for tree construction, $O(n)$ for encoding/decoding
- Trie operations: $O(m)$ where m is string length

- Space-time tradeoffs: Tries use more space but provide faster lookup than hash tables for prefix operations

Real-World Applications:

- Search engines use pattern matching for query processing and indexing
- Compression algorithms reduce storage and transmission costs
- Bioinformatics relies on sequence alignment for DNA/protein analysis
- Autocomplete and spell-check systems use tries for fast prefix matching
- Version control systems use text differencing algorithms

These algorithms and data structures provide the foundation for advanced text processing applications and continue to be relevant as the volume of digital text grows exponentially.

13.8 Solutions/Answers to Check Your Progress

1. Question: What is the time complexity of the brute force pattern matching algorithm? a) $O(n + m)$ b) $O(n \times m)$ c) $O(n \log m)$ d) $O(m \log n)$

Answer: b) $O(n \times m)$

Explanation: The brute force algorithm checks every position in the text (n positions) and for each position, compares up to m characters of the pattern. In the worst case, this results in $O(n \times m)$ time complexity.

2. Question: What advantage does the KMP algorithm provide over brute force pattern matching? a) Uses less memory b) Avoids redundant character comparisons c) Works only with binary strings d) Requires preprocessing of the text

Answer: b) Avoids redundant character comparisons

Explanation: KMP algorithm preprocesses the pattern to create a failure function (LPS array) that allows it to skip redundant comparisons when a mismatch occurs, achieving $O(n + m)$ time complexity instead of $O(n \times m)$.

3. Question: In dynamic programming for edit distance, what do the three operations represent? a) Insert, delete, copy b) Insert, delete, substitute c) Match, substitute, transpose d) Copy, move, replace

Answer: b) Insert, delete, substitute

Explanation: The edit distance (Levenshtein distance) considers three basic operations: inserting a character, deleting a character, and substituting one character for another.

4. Question: What is the main principle behind Huffman coding? a) Assign equal-length codes to all characters b) Assign shorter codes to more frequent characters c) Use alphabetical ordering for code assignment d) Minimize the number of unique characters

Answer: b) Assign shorter codes to more frequent characters

Explanation: Huffman coding is a greedy algorithm that builds an optimal prefix-free code by assigning shorter binary codes to characters that appear more frequently in the text.

5. Question: What is the time complexity of searching for a word of length m in a trie? a) $O(\log m)$ b) $O(m \log n)$ c) $O(m)$ d) $O(n)$

Answer: c) $O(m)$

Explanation: Searching in a trie requires traversing from root to leaf following the characters of the word. Since we visit exactly m nodes (one for each character), the time complexity is $O(m)$.

6. Question: Which compression technique is most suitable for data with many consecutive repeated characters? a) Huffman coding b) Run-length encoding c) LZ77 d) Arithmetic coding

Answer: b) Run-length encoding

Explanation: Run-length encoding is specifically designed for data with consecutive repeated characters, representing them as character-count pairs (e.g., "AAAAA" becomes "A5").

7. Question: What is the space complexity of computing edit distance using dynamic programming? a) $O(1)$ b) $O(m)$ c) $O(n)$ d) $O(m \times n)$

Answer: d) $O(m \times n)$

Explanation: The standard DP approach for edit distance creates a 2D table of size $(m+1) \times (n+1)$ to store intermediate results, resulting in $O(m \times n)$ space complexity.

8. Question: In a compressed trie (PATRICIA tree), what happens to nodes with only one child? a) They are deleted b) They are merged with their parent c) They are merged with their child d) They remain unchanged

Answer: c) They are merged with their child

Explanation: Compressed tries reduce space by merging nodes that have only one child with their child node, storing the edge labels to maintain the original string structure.

9. Question: What is the main advantage of the Boyer-Moore algorithm? a) It works from left to right b) It can skip multiple characters when a mismatch occurs c) It uses constant extra space d) It works only with ASCII characters

Answer: b) It can skip multiple characters when a mismatch occurs

Explanation: Boyer-Moore algorithm uses the bad character rule and good suffix rule to skip multiple positions when a mismatch occurs, potentially achieving $O(n/m)$ time complexity in the best case.

10. Question: Which property makes Huffman codes uniquely decodable? a) All codes have the same length b) Codes are assigned alphabetically c) No code is a prefix of another code d) Codes use only 0s and 1s

Answer: c) No code is a prefix of another code

Explanation: Huffman codes are prefix-free, meaning no code word is a prefix of another. This property ensures that the encoded text can be uniquely decoded without ambiguity.

13.9 Further Readings

- "Algorithms on Strings, Trees, and Sequences" by Dan Gusfield
- "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein
- "Pattern Recognition and Machine Learning" by Christopher Bishop
- "Compilers: Principles, Techniques, and Tools" by Aho, Sethi, and Ullman

Online Resources:

- Stanford CS97SI: Introduction to Computational Biology
- MIT 6.046J: Design and Analysis of Algorithms

- Coursera: Algorithms on Strings Specialization
- GeeksforGeeks: String Algorithms and Data Structures

Linked List

Research Papers:

- "A Fast String Searching Algorithm" by Boyer and Moore (1977)
- "Fast Pattern Matching in Strings" by Knuth, Morris, and Pratt (1977)
- "A Method for the Construction of Minimum-Redundancy Codes" by Huffman (1952)
- "A Universal Algorithm for Sequential Data Compression" by Ziv and Lempel (1977)

Software Tools:

- BLAST for biological sequence alignment
- Apache Lucene for text search and indexing
- Various compression utilities (gzip, bzip2, 7zip)
- Text editors with advanced search capabilities



UNIT 15 MEMORY MANAGEMENT AND B-TREES

Structure

- 15.1 Introduction
- 15.2 Objectives
- 15.3 Memory Management
 - 15.3.1 Kinds of Data storage
 - 15.3.2 Memory Management in Python
 - 15.3.3 Memory Fragmentation
 - 15.3.4 Garbage Collection
 - 15.3.5 Reference Counting and Cycle Detection
- 15.4 Memory Hierarchies and Caching
 - 15.4.1 The Memory Hierarchy
 - 15.4.2 Caching Strategies
- 15.5 External Searching and B-Trees
 - 15.5.1 Why do we need external searching and B-Tree
 - 15.5.2 Structure of a B-Tree
 - 15.5.3 Search in a B-Tree
 - 15.5.4 Insertion in a B-Tree
 - 15.5.5 Deletion in a B-Tree
 - 15.5.6 Applications of B-Trees
- 15.6 External-Memory Sorting
 - 15.6.1 External Merge Sort
 - 15.6.2 External Sorting Techniques
 - 15.6.3 Complexity of External Sorting
- 15.7 Summary
- 15.8 Solutions/ Answers
- 15.9 Further Readings

15.1 Introduction

Efficient use of memory is one of the central concerns of designing algorithms and implementing it in any programming language. The main concern of the programmer is to design programs such that they use storage efficiently. In high-performance applications and systems handling large datasets, the way data is stored, accessed, and manipulated has a direct impact on execution speed.

The chapter introduces the concept of memory management in Python, highlighting mechanisms such as heap allocation, reference counting, and garbage collection that simplify memory handling. It then introduces the concept of memory hierarchies and caching, explaining how layered memory organization and locality of reference reduce access time and improve system performance through effective caching and replacement strategies.

This chapter further presents, B-Trees, an essential data structure for managing large sorted datasets and are commonly used in databases and file systems. B-Trees are discussed as efficient solutions for external searching reducing disk input/output operations. Searching, insertion and deletion in b-tree are explained with examples. The chapter also examines external sorting techniques, with particular emphasis on external merge sort, as practical methods for organizing large datasets stored on

15.2 Objectives

After going through this unit, you should be able to

- Understand the principles of memory management in Python and modern computer System;
 - Explain memory hierarchies and caching mechanisms
 - Explain the structure and operations of B-Trees, and
 - Apply principles of external-memory sorting, such as external merge sort.
-

15.3 Memory Management

Memory management is the process of organizing and controlling how memory is used in a computer system. It is a major concern of programming language and algorithm designers. Each algorithm designer controls the use of computer memory during program execution, including allocating, using, and freeing memory space.

15.3.1 Kinds of Data storage

Data storage during program execution can be broadly classified into primary (main memory) and secondary (external storage). Python primarily works with both through built-in mechanisms.

1. **Global or Static Storage:** In python to have global storage there are some global variables which are declared at the beginning of the program. Static variables are declared inside the function.

Example: `count = 0` # Global variable

```
def increment():
    global count
    count += 1
increment()
increment()
print(count)
```

Here, `count` remains stored for the full lifetime of the program

2. **Stack Storage:** It is a local storage that is allocated on a run time. It is used in a function or a method. Stack storage is used during execution of function calls. For example, passing parameters to function, local variables, compiler-generated temporaries (i.e. for expression evaluation) etc.

Example: `def add(a, b):` # `a` and `b` stored in stack

```
result = a + b    # result stored in stack
return result
x = 10
y = 20
print(add(x, y))
```

In this example, `a`, `b`, and `result` are created in stack memory during function execution and destroyed after return.

3. **Heap Data:** Heap is used to allocate and deallocate the memory at arbitrary times during execution. Complex storage management algorithms are needed in heap. All Python objects such as: Lists, Dictionaries, Class objects are stored in the heap.
4. **Constant Storage:** it is a storage of Literal pool. This includes string literal, Numeric Constant, Boolean values.

```
a = 100
```

```
b = 100
```

```
print(a is b) # True (same memory location)
```

Here, python reuses small immutable constants.

15.3.2 Memory Management in Python

Python uses an automatic memory-management system that handles allocation and deallocation. The main components are:

1. **Memory Heap or Python Heap:** In Python, all objects are created and stored in a common memory area known as the Python heap. Whenever a statement such as `w = Widget()` is executed—where `Widget` represents a user-defined class—a new object of that class is dynamically created and allocated space within the heap memory.

The responsibility for requesting memory from the operating system, allocating space for objects, and efficiently managing the Python heap during program execution lies entirely with the Python interpreter. This automatic memory management allows programmers to focus on program logic rather than low-level memory handling.

2. **Memory Manager:** Memory Manager controls the allocation and release of memory blocks within the heap. This can be done with the help of Memory Fragmentation techniques.
3. **Garbage Collector:** A system that automatically frees memory used by objects that are no longer needed.

The Python interpreter communicates with the operating system to request memory and manages this memory internally using optimized allocation strategies such as memory pools and arenas.

15.3.3 Memory Fragmentation

In heap-based memory allocation, the available memory is organized into contiguous blocks and managed using a linked list structure known as the free list. As memory blocks are repeatedly allocated and released, the pattern of free space continuously changes, causing the unused memory to be split into disjoint holes separated by allocated regions. This breaking up of free memory into separate holes is referred to as memory fragmentation.

Fragmentation is of two types: internal fragmentation, which occurs when allocated memory contains unused space because the reserved block is larger than the actual requirement, and external fragmentation, which occurs when free memory is scattered into many small non-contiguous blocks across the heap, making large memory allocations difficult. In figure1(a), it shows that allocated space to Process, P1 is 500 MB, But P1 utilizes only 25 MB Space and other space is wasted, cannot be used by another process. Figure 1(b) shows that if a process need 500MB, but this space is available but not as a continuous memory space.

In Python, fragmentation mainly arises due to frequent creation and deletion of objects, variable object sizes, and the dynamic resizing of data structures such as lists and dictionaries. To minimize the effects of fragmentation, Python employs efficient techniques such as object pooling, specialized small-object allocators, and automatic garbage collection.

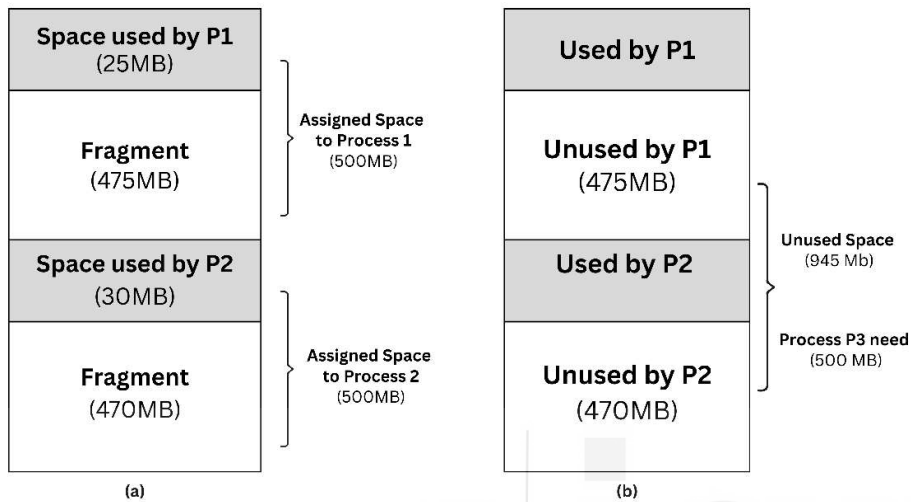


Figure 15.1: Internal and External Fragmentation

Various memory allocation strategies, such as first-fit, best-fit, next-fit, and worst-fit, are used to manage free memory blocks in the heap. The best-fit algorithm selects the smallest available block that can satisfy a request, which may lead to a higher degree of external fragmentation due to the creation of many small leftover gaps. The first-fit algorithm allocates the first suitable block it encounters, which often results in fragmentation accumulating near the beginning of the free list. The next-fit strategy continues the search from the point of the previous allocation, thereby distributing fragmentation more evenly across memory, but it can make large contiguous allocations harder to obtain over time. The worst-fit algorithm, on the other hand, allocates memory from the largest available block, with the intention of preserving sizable continuous free regions to reduce fragmentation. The selection of an appropriate allocation algorithm involves a trade-off between allocation speed, memory utilization, and the level of fragmentation produced.

15.3.4 Garbage Collection

In programming languages such as C and C++, the responsibility of explicitly releasing memory occupied by objects lies with the programmer. This task is often neglected by beginners and can lead to serious memory-related errors, such as memory leaks and dangling pointers, even in the hands of experienced developers. In contrast, the designers of Python shifted the entire responsibility of memory management to the Python interpreter. The automatic process through which unused or “stale” objects are identified, the memory occupied by them is released, and the reclaimed space is returned to the free list is known as garbage collection.

To enable automatic garbage collection, the system must first be able to identify those objects that are no longer needed. Since the Python interpreter cannot practically examine the full logical meaning of an arbitrary program, it applies a conservative reference-based rule to determine whether an object can be reclaimed. For a program to access an object, there must exist either a direct or an indirect reference to that object; such objects are classified as live objects.

A direct reference exists when an object is associated with an identifier in an active namespace, such as the global namespace or the local namespace of a currently executing function. For example, immediately after the statement, `w = Widget( )` is executed, the identifier `w` becomes a direct reference to the newly created object. Objects that have at least one direct reference are called root objects.

An indirect reference occurs when an object is referenced within the internal state of another live object. For instance, if the widget object contains a list as one of its attributes, that list is also considered live because it can be accessed indirectly through the reference `w`. This definition of live objects is recursive—any objects referenced within that list are likewise classified as live.

The Python interpreter assumes that all live objects are currently in use and must therefore be preserved in memory. Any objects that are not reachable through either direct or indirect references are considered unused and are eligible for garbage collection.

15.3.5 Reference Counting and Cycle Detection

Each Python object maintains an integer known as its reference count, which indicates the total number of references to that object currently present in the system. Python relies primarily on this reference-counting mechanism to manage object lifetimes. The reference count is increased whenever a new reference to the object is created and decreased whenever an existing reference is redirected to another object. When an object's reference count reaches zero, the object is no longer considered live, and the memory allocated to it is automatically reclaimed.

Maintaining a reference count for each object requires only constant additional space, $O(1)$, per object, and the operations of incrementing and decrementing this count also take constant time, $O(1)$, per update. The Python interpreter provides a way for a running program to inspect an object's reference count through the `sys` module, which includes the function `getrefcount( )`. This function returns an integer value representing the current number of references to the object passed as its argument.

Cycle detection addresses situations in which objects continue to reference one another but are no longer accessible from any root object, making them unreachable and therefore eligible for garbage collection. A nonzero reference count alone does not guarantee that an object is still live, because objects may exist in tightly connected groups that reference each other while remaining completely detached from the set of root objects. For example, a list may retain a positive reference count through internal links, yet become eligible for garbage collection once the external identifier that referenced it goes out of scope. To handle such scenarios, Python periodically employs advanced garbage-collection techniques, such as the mark-and-sweep algorithm, particularly when available memory becomes limited.

In the mark-and-sweep garbage collection algorithm, each object contains an associated mark bit that indicates whether it is active or inactive. When a garbage-collection cycle begins, all mark bits are initially reset. The algorithm then identifies all root objects and recursively traverses their references using a depth-first search, marking every reachable object as live; this process constitutes the mark phase. After marking is complete, the algorithm scans the entire heap and reclaims the memory occupied by all unmarked objects. During this sweep phase, the algorithm may also merge adjacent free blocks to temporarily reduce external fragmentation.

However, reference cycles (for example, self-referencing structures) can prevent cleanup. Python's garbage collector detects and frees such cyclic references.

15.4 Memory Hierarchies And Caching

With the widespread adoption of computing across all sectors of society, modern software applications are required to handle extremely large volumes of data. Such applications include online financial transaction processing, database organization and maintenance, and the analysis of customers' purchasing behavior and preferences. In many of these cases, the volume of data is so high that the overall system performance often depends more on the time required to access the data than on the raw processing speed of the CPU.

15.4.1 The Memory Hierarchy

Memory Hierarchy is the structured arrangement of various storage types that differ significantly in speed, capacity, cost, and proximity to the CPU. The primary goal of this hierarchy is to provide the illusion of a large, fast, and inexpensive memory system, even though physical limitations prevent any single memory technology from having all three characteristics simultaneously.

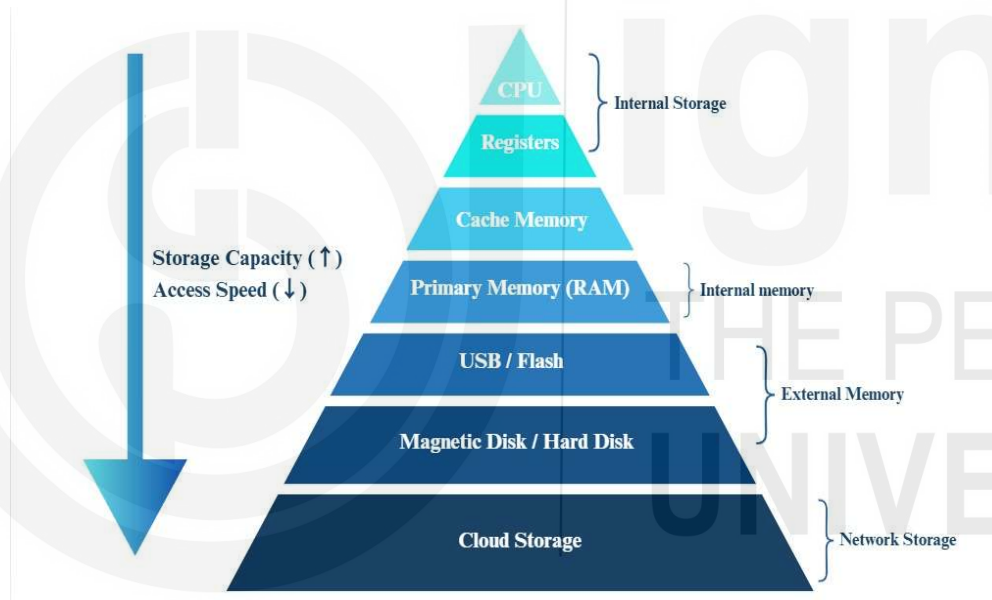


Figure15.2:Memory Hierarchy

A view of memory hierarchy is shown in Figure 15.2. It includes the following layers:

1. *Registers*: They are the nearest to the CPU, holding values needed for immediate operations. Their access to CPU is fastest, but they are small in storage space and expensive.
2. *Cache* – Small, high-speed memory located close to or within the CPU to reduce access time for frequently used data.
3. *Internal Memory (RAM)* – It has a medium-speed access to CPU, larger in capacity than register and cache memory. It is used to store actively running programs and data.
4. *External Memory*: They are the secondary storage devices with high capacity for long-term data retention but with slow access speeds. Their capacity can be increased or decreased based upon the requirement.

5. *Tertiary/Cloud Storage* – Massive, low-cost storage layers accessed infrequently or for backups.

CPU processes the data, it takes less time to fetch data through Internal memory, accessing data through external memory is expensive; disk I/O operations takes milliseconds while CPU processes data on the order of nanoseconds, one million times faster.

15.4.2 Caching Strategies

A cache stores recently or frequently accessed data to reduce latency. Whenever CPU needs the data it first checks the Cache memory. If the data is found (a cache hit), the access is very fast. If not found (a cache miss), the system must retrieve the data from lower memory levels, which is slower. Caching works on two key principles:

1. *Temporal locality*: When program elements accesses a specific memory address, it is more likely to revisit that same address again in the near future. Example, a counter variable may be referenced repeatedly—both when its value is used in expressions and when it is updated. Computer architects often summarize this behavior with the observation that a small portion of a program (around 10%) accounts for the vast majority of its execution time (about 90%).

2. *Spatial locality*: When program elements accesses one memory address, there is an increased chance that it will soon access nearby addresses as well. A common example is array processing, where programs typically read or write elements one after another, resulting in access patterns that follow a predictable, sequential flow.

To take advantage of temporal locality, it is useful to keep recently accessed web pages in a cache. Doing so allows the system to fetch these pages much more quickly if they are requested again shortly afterward. Caches are managed using hardware-controlled algorithms that decide which web pages should be loaded into the cache, when older web pages should be removed to make room for new entries, and how consistency should be maintained across multiple cache levels. To accomplish this efficiently, systems rely on various page replacement policies. Some of the policies are:

1. Least Recently Used (LRU), which evicts the page that has not been accessed for the longest time.
2. First-In, First-Out (FIFO), which removes the oldest web page entry in the cache
3. Least Frequently Used (LFU), which discards the web page that has been accessed the fewest times.
4. Random Replacement, which chooses a web page entry at random to be removed.

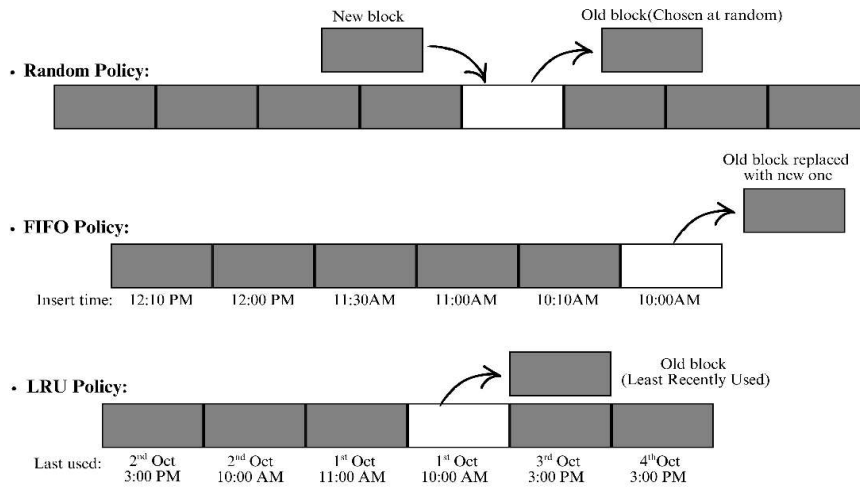


Figure15.3: Page Replacement Policies

Algorithms and data structures that access contiguous blocks of memory (arrays, B-Trees) perform better than pointer-heavy structures (linked lists) because they take advantage of caching.

Python provides software-level caching mechanisms that improve performance by avoiding repeated computation or data retrieval.

One common caching tool in Python is `functools.lru_cache`, which stores the results of expensive function calls.

Program 15.1 gives a function to compute factorial of a number in python using software caching

```
from functools import lru_cache
@lru_cache(maxsize=256)
def compute_factorial(n):
    if n <= 1:
        return 1
    return n * compute_factorial(n - 1)
print(compute_factorial(10))
```

Program 15.1: Program to compute factorial of a number showing use of software caching

This implementation ensures that repeated calls with the same argument are served from the cache, reducing computation time.

☞ Check Your Progress – 1

- 1) What is the role of the garbage collector in Python?
- 2) How does caching improve performance?
- 3) Why is memory management important?
- 4) What is fragmentation?

15.5 External Searching And B-Trees

This module presents external searching and B-tree. When data cannot fit entirely in main memory, external searching techniques are used to minimize disk accesses. B-Trees are a fundamental data structure for this purpose.

15.5.1 Why do we need external searching and B-Tree

- All the data structures discussed in this book so far favor data stored in internal memory and hence the retrieval of the data is fast. When the volume of the data is enormous, it is stored in database programs. This data is too large to fit in main memory so it is stored in disk or tapes.
- Disk access is very expensive; it takes more time to process than the processing time of CPU.
- Balanced binary search trees (AVL & Red-Black tree) have good performance if the entire data can fit in the main memory.
- These trees are not optimized for external storage and require many disk accesses, that gives poor performance for a very large data.
- The Disk accesses can be reduced, if the tree height is reduced. This can be done by increasing the number of children of each node (instead of only two child nodes). It can also be done if it stores multiple records in a block on the disk.

To achieve the above goal, we use B-trees which is a generalization of Binary Search tree. Unlike binary trees, B-Trees are designed to minimize disk reads by grouping keys into nodes that match the size of a typical disk block. This design enables fast search, insertion, and deletion, even for datasets too large to fit into main memory. Many databases and filesystems use B-Trees or their variants for indexing and data organization.

15.5.2 Structure of a B-Tree

A B-Tree of order m is a self-balancing search tree designed so that all leaf nodes appear at the same depth, ensuring that the tree always remains height-balanced. Each node contains multiple keys, and the number of children it may have lies within specific bounds. The goal is to prevent nodes from having empty subtrees and to maintain efficient access even when the dataset is too large to fit in main memory.

A B-Tree of order m is characterized by the following properties:

1. Each node can contain multiple keys stored in sorted order.
2. A B-Tree of order m permits no more than m children for any node.
3. Every internal node, except the root, maintains at least $\lceil m/2 \rceil$ children to satisfy the minimum occupancy requirement.
4. The root node contains either all keys (when the tree is a single leaf) or has at least two children in nontrivial configurations.
5. The maximum number of keys that a node may contain is $m - 1$.
6. Internal nodes other than the root must hold at least $\lceil m/2 \rceil - 1$ keys to preserve balance and density.
7. The height of the B-Tree grows or contracts only from the root, eliminating the need for structural rotations.
8. The placement of keys within each node supports efficient multiway search, reducing the number of disk or memory accesses.

Figure 15.4 depicts a B-tree of order 4, each node contains $m-1$ keys, so here maximum number of keys can be three in each node. Internal nodes hold at least 1 key to preserve balance.

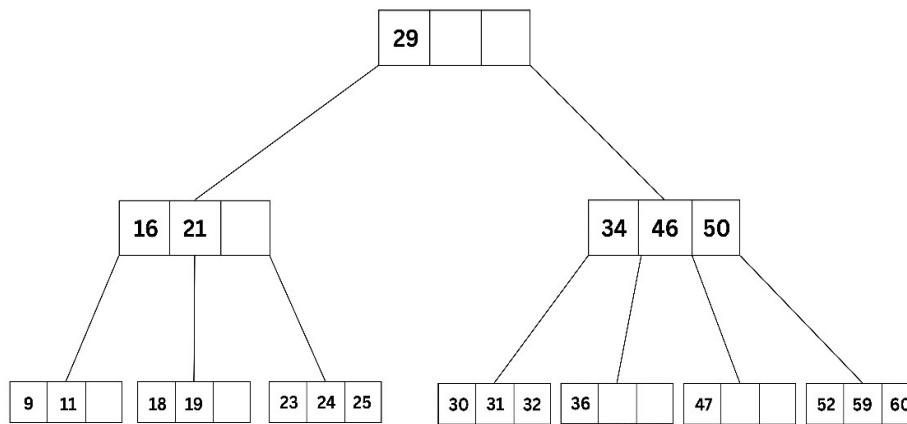


Figure 15.4:Representation of a B-tree of order 4

15.5.3 Search in a B-Tree

Searching in a B-Tree involves comparing keys to find the correct node and then traversing child pointers. Searching follows the idea as searching in a binary search tree (BST) i.e., keys on the left of a node are smaller and keys on the right are greater. However, instead of having only two children like a BST, a B-Tree node may contain multiple keys and multiple child pointers.

Insertion ensures the tree remains balanced by splitting nodes when full. The keys in each B-Tree node are stored in sorted order, keys smaller than the first key go to the leftmost child, keys between successive keys follow the corresponding internal children, and keys larger than the last key follow the rightmost child.

The search follows an alternating two-step procedure that begins at the root of the B-tree:

1. Perform a Binary search within the current node on the sorted keys stored in that node.
 - a. If a key matching the search value is found, the corresponding record is returned.
 - b. If the key is not found and the current node is a leaf, the search is unsuccessful because no further paths remain.
2. If the node is an internal node, the appropriate child pointer is selected based on the key comparisons, and the search continues recursively in that subtree.

For example, consider searching for a key with value 32 in the B-tree shown in Figure 15.4. The algorithm begins at the root node, where the comparisons lead to selecting the second (right) child. At the next level, the keys are examined again, causing the search to follow the first child pointer. This leads to a leaf node that contains the record with key value 32, and the search terminates successfully. Program 15.2 gives a simple Python method illustrating the search logic inside a B-Tree node.

```
class BTreeNode:
```

```
    def __init__(self, t, leaf=False):
```

```
        self.t = t
```

```
    self.leaf = leaf
```

```
    self.keys = [ ]
```

```
    self.children = [ ]
```

```
    def search(self, k):
```

```
        i = 0
```

```
        # Find the first key >= k
```

```

while i < len(self.keys) and k > self.keys[i]:
    i += 1
# If the found key is equal to k, return this node
if i < len(self.keys) and self.keys[i] == k:
    return self
# If this is a leaf, the key is not found
if self.leaf:
    return None
# Otherwise search in the appropriate child
return self.children[i].search(k)

```

Program 15.2: Searching in a B-Tree

15.5.4 Insertion in a B-Tree

Insertion in a B-Tree follows a well-defined procedure that preserves all structural and balance properties of the tree. The process ensures that no node exceeds its maximum capacity and that all nodes (except the root) maintain the required minimum number of keys. A new value is inserted into the B-Tree using the following algorithm:

1. Search the B-Tree to identify the correct leaf node where the key value should be placed.
2. If the leaf node is not full, insert the new value into the node in sorted order.
3. If the leaf node is full, it means it contains $m-1$ key values, then split the node:
 - a. Insert the new key in order into the existing set of keys.
 - b. Identify the median key.
 - c. Divide the node at the median key into two separate nodes.
 - d. Move the median key to the parent node. If the parent node becomes full as a result, recursively apply the split operation to the parent.
 - e. If the root node splits, create a new root node containing the promoted median key.
 - f. The insertion finishes once no further splits are required.

Note that The insertion procedure guarantees that each node maintains a minimum occupancy of half its capacity.

Example 1: Look at the B-Tree given in Figure 15.4, Insert element 14 and element 33.

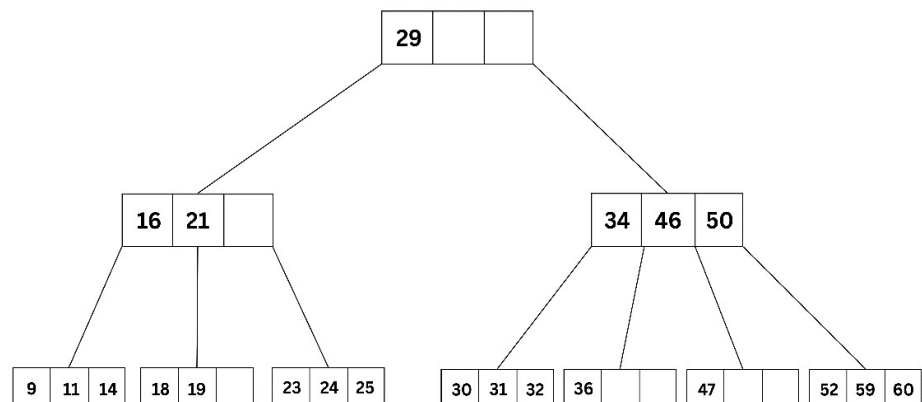


Figure 15.5: Insertion of element '14' in B-tree

In Figure 15.5, we have inserted element 14 in the tree at its appropriate position in sorted order. It is easily inserted as the leaf nodes are not full. But when we try to insert 33, it cannot be inserted directly at its position because the node full and contains three values. In this case, we need to split the node to form two different nodes. But before dividing, key values (Including the new value) are arranged in sorted order. The new order is 30,31,32 and 33. The median node value is 31, so move 31 into parent node and divide the leaf node. In this case parent node is also full and all its child nodes are already created. Split procedure will be applied here also. The new order at the parent node is 31,34,46 and 50. The median node value here is 34, so move 34 into its parent's node. The new tree is depicted in Figure 15.6.

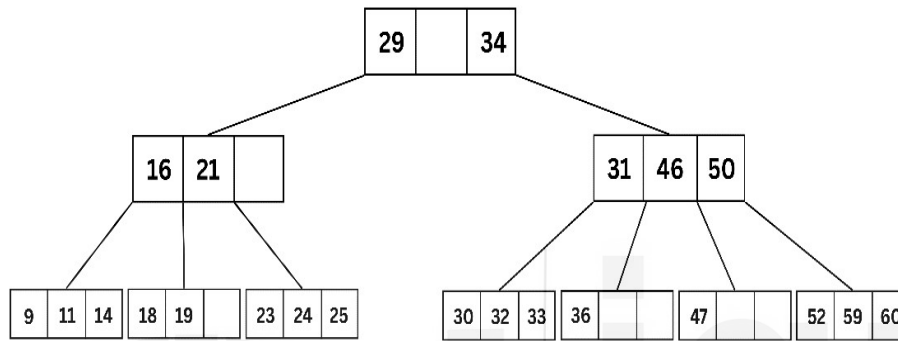


Figure 15.6: Insertion of another element '33' in a B-tree

The python code for the B-Tree Insertion is given below as Program 15.3

```
class BTreeNode:
    def __init__(self, t, leaf=False):
        self.t = t          # Minimum degree (order)
        self.leaf = leaf    # True if node is a leaf
        self.keys = []      # List of keys
        self.children = []  # List of child pointers

class BTree:
    def __init__(self, t):
        self.t = t
        self.root = BTreeNode(t, leaf=True)

    def insert(self, key):
        root = self.root
        if len(root.keys) == 2 * self.t - 1: # If root is full, tree grows in height
            new_root = BTreeNode(self.t, leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
            self._insert_non_full(new_root, key)
        else:
            self._insert_non_full(root, key)

    def _insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            # Insert key into the correct position in leaf
            node.keys.append(None)
            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1
            node.keys[i + 1] = key
```

```

else# Find the child that should receive the key
    while i >= 0 and key < node.keys[i]:
        i -= 1
        i += 1
        # If the child is full, split it
        if len(node.children[i].keys) == 2 * self.t - 1:
            self._split_child(node, i)
            if key > node.keys[i]:
                i += 1
            self._insert_non_full(node.children[i], key)
        def _split_child(self, parent, index):
            t = self.t
            full_child = parent.children[index]
            new_child = BTreeNode(t, leaf=full_child.leaf)
            parent.keys.insert(index, full_child.keys[t - 1])# Median key moves up to the parent
            parent.children.insert(index + 1, new_child)
            # Split keys
            new_child.keys = full_child.keys[t:]
            full_child.keys = full_child.keys[:t - 1]
            if not full_child.leaf:# Split children if not leaf
                new_child.children = full_child.children[t:]
                full_child.children = full_child.children[:t]
            def traverse(self, node=None):
                if node is None:
                    node = self.root
                for i in range(len(node.keys)):
                    if not node.leaf:
                        self.traverse(node.children[i])
                print(node.keys[i], end=" ")
                if not node.leaf:
                    self.traverse(node.children[-1])

```

Program 15.3: Program to Implementation of B-Tree Insertion\

To use the above code, you can call the Insert() Function and traverse () function as follows:

```

btree = BTree(t=3)

values = [10, 20, 5, 6, 12, 30, 7, 17]
for v in values:
    btree.insert(v)

print("B-Tree traversal:")
btree.traverse()

```

15.5.5 Deletion in a B-Tree

Deletion in a B-Tree is done from leaf node. As the first step, a leaf node is deleted and in the next step an internal node is deleted. The steps involved in deleting a leaf node given below:

1. Search the B-Tree to locate the leaf node that contains the key to be deleted.
2. If the leaf node contains more than the minimum required number of keys (that is, more than $\lceil m/2 \rceil - 1$ keys), simply delete the key from the node.
3. If the leaf node contains fewer than the minimum required number of keys (that is, fewer than $\lceil m/2 \rceil - 1$ keys), the node must be rebalanced by borrowing keys from its siblings or by merging nodes..
 - a. If the left sibling contains more than the minimum number of keys, transfer the largest key from the left sibling to the parent node, and move the corresponding separator key from the parent down into the

leaf node from which the deletion is being performed..

- b. Otherwise, if the right sibling contains more than the minimum number of keys, transfer the smallest key from the right sibling to the parent node, and move the corresponding separator key from the parent down into the leaf node where the deletion occurs
- c. If both the left and right siblings contain only the minimum number of keys, merge the leaf node with one of its siblings along with the separating key from the parent node to form a single leaf node. The merged node must not exceed the maximum allowed number of keys ($m - 1$). If the removal of the separator key causes the parent node to contain fewer than the minimum number of keys, the same rebalancing procedure is applied recursively to the parent. This process may propagate upward and, in some cases, result in a reduction in the height of the B-Tree.

Note: To delete a key from an internal node, replace the key with either its in-order predecessor or successor. Since this predecessor or successor always resides in a leaf node, the deletion process ultimately reduces to deleting a key from a leaf node.

Example 1: Look at the B-Tree given in Figure 15.4 of order 4, Delete element 25, then element 36 and element 46.

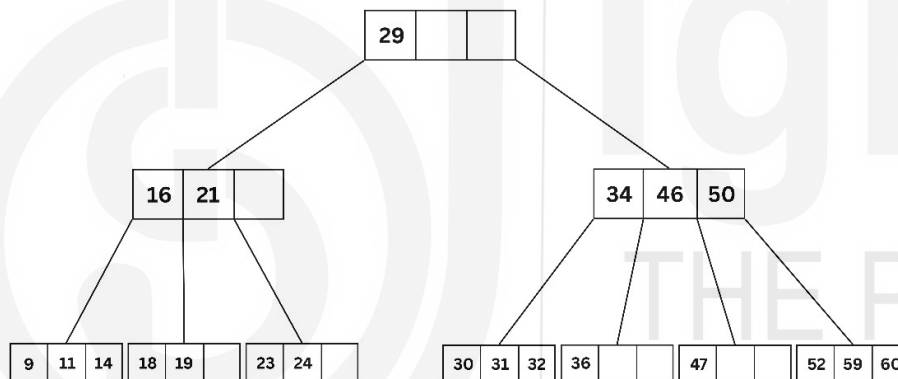


Figure 15.7: Deletion of element '25' in B-tree

When we search for element 25, it is found to be a leaf node. This leaf node contains 3 keys, which is more than the minimum required (for a typical B-Tree of order 4, minimum keys = 1 or 2 depending on convention). Therefore, simply delete it. Figure 15.7 shows the tree after this deletion.

Now consider the deletion of the key 36. The key is located in a leaf node that contains only the minimum permissible number of keys; therefore, deleting it directly would result in an underflow. To resolve this, the left and right sibling nodes are examined to determine whether they contain more than the minimum number of keys. In this case, the left sibling satisfies this condition. Consequently, the largest key (32) from the left sibling is promoted to the parent node, and the corresponding separator key (34) from the parent is moved down into the deficient leaf node. After this redistribution, the key 36 can be safely removed from the leaf node without violating the B-Tree properties. The new formed tree is shown in Figure 15.8.

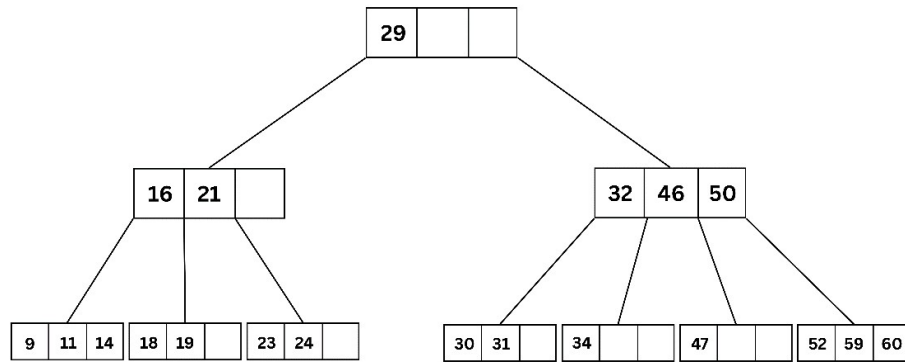


Figure 15.8: Deletion of element '36' in B-tree

Now, after tree formed in figure 15.8, delete element 46. Search for element 46. It is found to be an internal node. Here, replaces the key with either its inorder predecessor, or its inorder successor (47). 47 is located as leaf node, so move it up. Move the smallest key (52) from the right sibling up to the parent. Move the separator key (50) from the parent down to the empty leaf. Figure 15.9 shows the final tree after deleting element 46 from it.

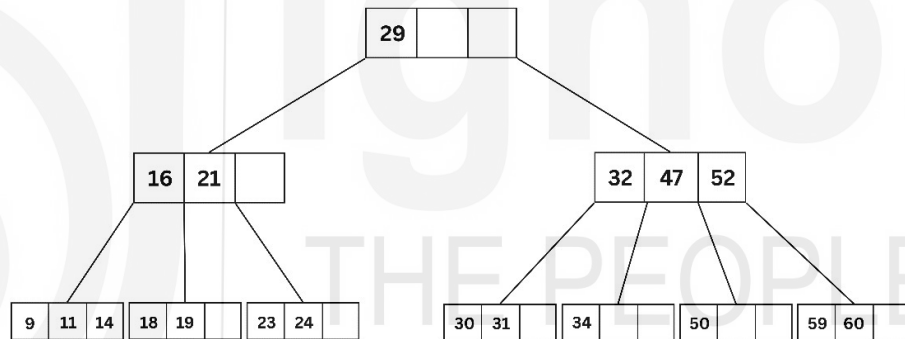


Figure 15.9: Deletion of element '46' in B-tree

15.5.5 Applications of B-Trees

B-Trees are widely used in systems that must manage large volumes of data efficiently, especially where disk access time is a critical factor. Their ability to minimize disk reads by storing many keys within a single node makes them ideal for external-memory indexing. As a result, B-Trees form the foundation of numerous database engines, such as MySQL and PostgreSQL, where they support fast insertion, deletion, and range queries. They are also integral to modern file systems including NTFS, HFS+, and ext4 which rely on them to organize directory structures and metadata. More broadly, B-Trees are employed in applications that require scalable indexing of substantial datasets, enabling quick and reliable access even when the data exceeds main memory capacity. The main applications are:

- Used in relational database indexing.
- Manage directory and metadata structures in file systems
- Support search and retrieval in large-scale data storage systems

15.6 External-Memory Sorting

In this section, we will discuss that there are many practical applications where

datasets are so large that they cannot fit entirely into the internal (primary) memory of a computer. In such cases, traditional internal sorting techniques such as Quick Sort, Bubble Sort, Heap Sort, Radix Sort, and similar algorithms cannot be applied directly. Sorting must instead be performed using external-memory sorting techniques, where data resides on secondary storage such as hard disks. In external sorting, data is transferred between disk and main memory in fixed-size units called blocks. A block consists of several records and represents the smallest unit of data that can be read from or written to disk at a time.

For a disk, there are three factors contributing to the read/write time:

- *Seek Time* – The time required for the disk's read/write head to move to the correct track where the data is located.
- *Rotational Latency* – The time taken for the disk platter to rotate so that the desired sector aligns with the read/write head.
- *Transmission Time* – The time needed to actually read or write the data once the head is positioned over the correct sector.

15.6.1 External Merge Sort

This large dataset requires external sorting techniques, such as the External Merge Sort. This method consists of two distinct phases:

Phase 1: Run Generation or Sorting Phase:

In the first phase, the large dataset residing on secondary storage is partitioned into smaller units known as runs, each of which is small enough to fit into main memory. These blocks are read into memory one at a time, sorted using an efficient internal sorting technique such as Quick Sort or Merge Sort, and then written back to disk as individual sorted runs. This process is repeated until the entire dataset has been processed, producing multiple sorted runs stored on secondary storage.

Phase 2: Merge Phase

In the second phase, the sorted runs produced during the first phase are merged to generate a single sorted output file. A multi-way merge strategy is typically employed, in which multiple runs are read concurrently into memory buffers, and the smallest elements among them are repeatedly selected and written to the output file. When the number of runs exceeds the available memory buffers, the merging process is carried out in multiple passes. This procedure continues until all runs are combined into one fully sorted dataset. By dividing the sorting process into these two phases, External Merge Sort efficiently processes very large datasets while minimizing costly disk access operations.

Algorithm To understand the working of External Merge Sort

Assume that four tapes are available on secondary storage. Two Input Tapes -Ta1, Ta2 and two output Tapes -Tb1, Tb2.

Initially the file to be sorted which is there in External memory is assumed to be on input tape, Ta1

Let, N be the total number of records (to be sorted) is on Ta1 and M be the records that can fit in the memory.

Phase 1: Run Generation

1. Divide the file on Ta1 into blocks of size M.
2. Read a block of size M from Ta1 into main memory, sort it, and store it on output tape Tb1.
3. Read the next block, sort it, and store it on Tb2.

- Alternatively, we perform writing on Tb1 and Tb2.
- Each sorted block produced during the run generation phase is called a run.

Phase 2: Merging of Runs

During this phase, runs from tapes Tb1 and Tb2 are merged and the results are written alternately to tapes Ta1 and Ta2.

- Read the first run from Tb1 and the first run from Tb2 into main memory, merge them, and store the merged output on Ta1.
- Read the second run from Tb1 and the second run from Tb2, merge them, and store the result on Ta2.
- Read the third run from Tb1 and the third run from Tb2, merge them, and store the result on Ta1.
- Continue this process of reading alternating runs from Tb1 and Tb2, merging them in main memory, and writing the merged output alternately to Ta1 and Ta2 until all runs have been processed.

All records which were originally on Tape Tb1 and Tb2 are shifted to tape Ta1 and Ta2. We have Tb1 and Tb2 free at this stage. In the next step retrieve data from tape Ta1 and Ta2, perform merge and sorting, store the result alternatively on Tb1 and Tb2.

- The process is repeated until a single run is produced, resulting in the final sorted file.

15.6.2 External Sorting Techniques

Different external sorting techniques are used depending on the number of available tapes or input/output buffers. The commonly used external sorting techniques are:

- 2-way Merge Sorting:** In this technique, two output tapes are used to merge sorted runs. Only two runs are merged at a time in each pass.
- K-way Merge Sorting:** In k-way merge sorting, more than two output tapes (or buffers) are available. If k output tapes are used, then k runs can be merged simultaneously. For example, when three output tapes are used, it is referred to as three-way merge sorting.
- Poly-phase Merge Sorting:** Polyphase merge sorting improves efficiency by unevenly distributing runs across the available tapes and eliminating the need for empty (dummy) runs, thereby reducing the total number of merge passes.

Example 1: 2-Way Merge Sorting

Given: No. of Tapes: Ta1, Ta2, Tb1, Tb2

Data/Records (initially on Ta1): 17, 3, 29, 56, 24, 18, 4, 9, 10, 6, 45, 36, 11, 43

N = 14 (total number of records)

M = 3 (memory capacity)

K = 2, incase of 2-way merge sort, which means we have two output tapes

Phase 1: Run Generation

The file is divided into blocks of size $m = 3$, and each block is sorted internally before being written alternately to the output tapes Tb1 and Tb2.

Three records are first loaded into main memory, sorted, and written to tape Tb1.

For example, the records (17, 3, 29) are sorted as (3, 17, 29) and stored on Tb1,
resulting in:
Tb1: 3,17,29

Next, another set of three records is read into main memory, sorted, and written to tape
Tb2.

The records (56, 24, 18) are sorted as (18, 24, 56) and stored on Tb2, giving:
Tb2: 18,24,56

The following three records are then processed in the same manner and written to Tb1.
After sorting (4, 9, 10), the contents of Tb1 become:
Tb1: 3,17,29 | 4,9,10

Subsequently, the next group of three records (6, 45, 36) is read, sorted to (6, 36, 45),
and stored on Tb2, updating it to:
Tb2: 18,24,56 | 6,36,45

Finally, the remaining records (11, 43) are read into memory, sorted, and written to
Tb1.

Tb1: 3,17,29 | 4,9,10 | 11,43

Status after Phase 1: At the completion of the run generation phase, the sorted runs are
distributed as follows:

Tb1: 3,17,29 | 4,9,10 | 11,43 (three runs)

Tb2: 18,24,56 | 6,36,45 (two runs)

Ta1			
Ta2			
Tb1	3, 17, 29	4, 9, 10	11, 43
Tb2	18, 24, 56	6, 36, 45	

Phase 2: Merging of Runs

First Merge Pass (Run length = 3)

Source Tapes: Tb1(3,17,29 | 4,9,10 | 11,43) and Tb2(18,24,56 | 6,36,45)

Destination Tape: Ta1 and Ta2

In the first merge operation, the first run from Tb1 and the first run from Tb2 are read
into main memory, merged in sorted order, and the resulting sequence is written to
tape Ta1. The contents of Ta1 after this operation are:

Ta1: 3,17,18,24,29,56

Next, the second run from Tb1 and the second run from Tb2 are merged in a similar
manner and the merged output is stored on tape Ta2. The contents of Ta2 after this
merge are:

Ta2: 4,6,9,10,36,45

For third Run, no merging is required because Tb1 contains only two elements
(11,43), two runs for Ta1.

Ta1: 3,17,18,24,29,56 | 11,43

Ta1	3, 17, 18, 24, 29, 56	11, 43
Ta2	4, 6, 9, 10, 36, 45	
Tb1		
Tb2		

Second Merge Pass (Run length = 6)

Source Tapes: Ta1: (3,17,18,24,29,56 | 11,43) and Ta2 (4,6,9,10,36,45)

Destination Tape: Tb1 and Tb2

In this second merge pass, the first run from Ta1 and the first run from Ta2 are read
into main memory, merged in sorted order, and the resulting sequence is written to
tape Tb1. After this operation, Tb1 contains:

Tb1: 3,4,6,9,10,17,18,24,29,36,45,56

Since tape Ta2 does not contain a second run, no further merging is required in this pass, and the remaining run from Ta1 is copied directly to Tb2, resulting in:

Tb2: 11,43

At this stage, each tape contains exactly one run. The final step is to merge these two remaining runs to obtain the fully sorted file.

Ta1	
Ta2	
Tb1	3, 4, 6, 9, 10, 17, 18, 24, 29, 36, 45, 56
Tb2	11, 43

Final Merge Pass (Run length= 12)

Source Tapes: Tb1: (3,4,6,9,10,17,18,24,29,36,45,56) and Tb2 (11,43)

Destination Tape: Ta1 and Ta2

In this final merge pass, Read the run from Tb1 and the run from Tb2 into main memory, merge them in sorted order, and write the resulting sequence to tape Ta1.

Ta1: 3,4,6,9,10,11,17,18,24,29,36,43,45,56

Ta1	
Ta2	
Tb1	3, 4, 6, 9, 10, 17, 18, 24, 29, 36, 45, 56
Tb2	11, 43

The tape Ta1 now contains the fully sorted values, 3,4,6,9,10,11,17,18,24,29,36,43,45,56. This example demonstrates the complete working of Two-Way External Merge Sort, including run generation, multiple merge passes, and final output formation.

The 2-way merge sort requires $\left\lceil \log_2 \frac{N}{M} \right\rceil + 1$ (initial run-construction pass) passes. For instance, if we have 14 records to be sorted with internal memory capacity of 3 records, then we will need $\left\lceil \log_2 \frac{14}{3} \right\rceil + 1 = 4$ passes, which are described below.

Example 2: K-Way Merge Sorting

In k-way merge sorting, k output tapes are used, allowing k runs to be merged simultaneously in each pass. For example, when $k = 3$, three output tapes are available for merging. Increasing the number of tapes reduces the total number of merge passes required to sort the input file. Thus, k-way merge sorting can be viewed as an extension of two-way merge sorting, designed to improve efficiency by merging multiple runs in a single pass.

The k-way merge sort requires $\left\lceil \log_k \frac{N}{M} \right\rceil + 1$ (initial run-construction pass) passes. For instance, if we have 14 records to be sorted with internal memory capacity of 3 records and 6 tapes ($k = 3$), then we will need $\left\lceil \log_3 \frac{14}{3} \right\rceil + 1 = 3$ passes, which are described below.

Given: No. of Tapes: Ta1, Ta2, Ta3, Tb1, Tb2, Tb3
Data/Records (initially on Ta1): 17, 3, 29, 56, 24, 18, 4, 9, 10, 6, 45, 36, 11, 43
N = 14 (total number of records)
M = 3 (memory capacity)
K=2, incase of 2-way merge sort, which means we have two output tapes

Pass 1: Initial run-construction pass (length of run = 3)

Ta1		
Ta2		
Ta3	This tape is not in use.	
Tb1	3, 17, 29	6, 36, 45
Tb2	18, 24, 56	11, 43
Tb3	4, 9, 10	

Pass 2: (Length of run = 9)

Ta1	3, 4, 9, 10, 17, 18, 24, 29, 56	
Ta2	6, 11, 36, 43, 45	
Ta3	This tape is not in use.	
Tb1		
Tb2		
Tb3		

Pass 3: (Length of run = 14 because of last pass and there are only 14 records to be sorted)

Ta1		
Ta2		
Ta3	This tape is not in use.	
Tb1	3, 4, 6, 9, 10, 11, 17, 18, 24, 29, 36, 43, 45, 56	
Tb2		
Tb3		

As per the concept of k-way merge, we need 2k tapes. However, in some cases when the records to be sorted are less in number, it may be possible to sort the records with reduced number of tapes. For example, using 3-way merge (which requires 6 tapes), sorting of 14 records (with internal memory capacity of 3 records) can be done with 5 tapes.

In our example, it can be seen that tape Ta3 is available for sorting but it is not used in the sorting process, due to less number of records.

Example 3 (Disk Based example)

Consider a file containing 5500 records that must be sorted using a computer memory with an internal memory capable of sorting at most 850 records. The input list is maintained on disk and has a block length of 250 records, An additional disk is available for use as a scratch disk, while the original input disk is read-only.

During the run generation phase, blocks of data are read from the input disk into primary memory, sorted internally, and written back to the scratch disk as sorted

runs. Because each block contains 250 records, the input file consists of 22 blocks.

Number of blocks = $5500/250 = 22$ blocks.

Given that internal memory can accommodate up to 850 records, only three blocks (750 records) can be processed at a time. $850 \div 250 = 3.4 \Rightarrow 3$ blocks per run. Consequently, the algorithm produces seven initial sorted runs ($5500/850 = 6.47$ approx. 7 runs), six of which contain three blocks each, while the final run contains the remaining single block. Each run is read from the input disk, sorted in primary memory and then written back to scratch disk.

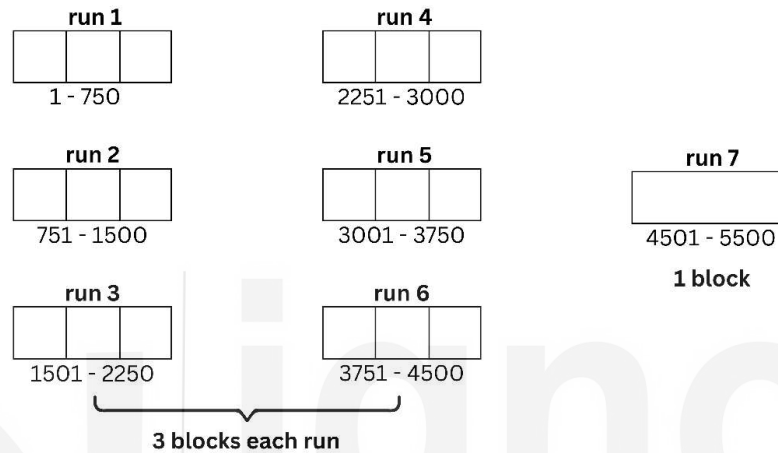


Figure 15.10: Blocks of data in each run-in run generation phase

In Merge phase, the sorted runs stored on the scratch disk are combined to produce the final sorted output. One block of memory is reserved as an output buffer (i.e., 250 records), and the remaining memory ($850 - 250 = 600$ records) allows for two input buffers ($600/250 = 2$), enabling a two-way merge. The runs are merged in successive passes, reducing the number of runs in each pass until only one fully sorted file remains. In this example, three merge passes are required to combine all seven runs into a single sorted sequence.

In pass 1: Run 1 merged with Run 2 gives Run 12, Run 3 merged with Run 4 gives Run 34, Run 5 merged with Run 6 gives Run 56, Run 7 remains.

In pass 2: Run 12 merged with Run 34 gives Run1234, Run 56 merged with Run 7 gives Run567.

In pass 3: Run1234 is merged with Run567

15.6.3 Complexity of External Sorting

The performance of external sorting is primarily determined by disk input/output operations, as disk access is significantly slower than CPU computation. Let N denote the total number of records, B the number of records per disk block, and M the capacity of main memory. The total number of disk blocks required to store the data is given by N/B .

During the run generation phase, each block is read once and written once, resulting in $2(N/B)$ disk I/O operations. In the merge phase, the entire dataset is read and written in each merge pass. If k merge passes are required, the I/O cost is $2k(N/B)$. Hence, the total I/O complexity of external sorting is: $2(1+k)N/B$.

Since the number of merge passes grows logarithmically with the number of runs, the overall complexity is:

$$O\left(\frac{N}{B} \log_{\frac{M}{B}} \frac{N}{B}\right)$$

In the example, the total number of records is $N = 5500$, the block size is $B = 250$ records, and the main memory can hold $M = 850$ records. Hence, the total number of disk blocks is: $N/B = 5500/250 = 22$ blocks. $2 * 22 = 44$ disk I/O operations will be performed. 3 merge phases are required since 3 merge passes are required, the merge phase incurs $2 \times 3 \times 22 = 132$ disk I/O operations. Therefore, the total disk I/O cost of the external sort is:

$44 + 132 = 176$ block I/O operations

This example demonstrates that the performance of external sorting is primarily governed by disk input/output operations rather than CPU processing time.

☛ Check Your Progress – 2

- 1) Draw a B-Tree with elements 40,10,20,80 of order 3.
- 2) In the above Tree Insert a) elements 70 and b) element 30.
- 3) In a B-Tree of order m , what is the maximum number of children that a node can have?
- 4) Name the most commonly used external sorting technique. _____.
- 5) What is the role of main memory in external sorting?
- 6) What type of disk access is preferred in external sorting?

15.7 Summary

This chapter highlights the use of memory management and data structures in determining overall computer system performance. Efficient utilization of memory resources was shown to be as important as algorithmic efficiency, particularly when handling large volumes of data. Python automatically manages memory using memory management techniques, such as heap allocation, reference counting, and garbage collection. These techniques were discussed as effective tools for simplifying memory handling while ensuring safe and efficient execution.

The chapter further explored memory hierarchies and caching, emphasizing how layered memory organization and locality of reference significantly reduce data access time. Various caching and replacement strategies were introduced to demonstrate how frequently accessed data can be retained in faster memory to enhance performance.

As data sizes grow beyond the capacity of main memory, the focus shifts to disk-friendly algorithms and data structures like B-Trees and external sorting. B-Trees were presented as an efficient solution for external searching, minimizing disk input-output through balanced structure and high branching factors. Additionally, external sorting techniques, particularly external merge sort, were analyzed as practical approaches for sorting large datasets that reside on secondary storage.

Together, the concepts presented in this chapter provide a strong foundation for designing scalable, high-performance systems capable of managing real-world, large-scale data efficiently.

15.8 Solutions / Answers

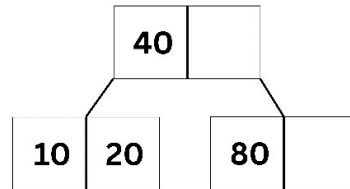
Check Your Progress 1

1. The garbage collector in Python automatically identifies and frees memory occupied by objects that are no longer in use, particularly handling reference cycles that cannot be reclaimed by reference counting alone.
2. Caching reduces average memory access time by storing frequently used data

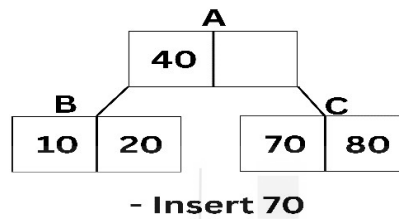
closer to the CPU.

3. It ensures efficient memory use and prevents memory leaks.
4. Fragmentation is wastage of memory due to unused gaps between blocks.

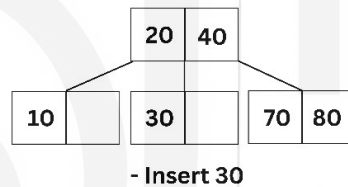
CheckYour Progress2



- 1.
2. (a)



- 2 (b)



3. m
4. Merge Sort
5. Main memory is used to hold manageable portions of the data for sorting and merging before writing the results back to disk.
6. Sequential disk access is preferred in external sorting because it is faster and more efficient than random access.

15.9 Further Readings

ReferenceBooks

1. Horowitz, E., Sahni, S., and Mehta, D. *Fundamentals of Data Structures in C++*. Universities Press (India) Pvt. Ltd., Hyderabad, 2008.
2. Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. *Data Structures and Algorithms in Python*. John Wiley & Sons, 2016.
3. Horowitz and Sahni, *Fundamentals of Computer Algorithms*, Computer Science Press, 2008